

# 基于随机模型预测控制的微网购电与储能调控优化

## 摘要

针对微网与外部电网的电力调控问题，本文以“供电可靠性为主、经济性为辅”为优化目标，在保证供电稳定、缺电风险可控的前提下，建立了从确定性线性规划到随机优化的递进式购电决策模型。

问题一在电价、负荷与光伏预测均已知（确定性）前提下，考虑储能容量与充放电功率约束，构建以购电费用最小为目标的线性规划模型，并融入储能“日循环”约束（0:00 与 24:00 储电量相同）。求解得到全天计划购电 59482.7 kWh，购电费 35126.8 元，充放电计划与储电量轨迹见正文表 3 与图 9。

问题二面对负荷与光伏逐时波动，引入“紧急购电（5 倍电价）”机制，建立两阶段随机规划模型，并以条件风险价值（CVaR）刻画购电费用的尾部风险。为消除“零缺电”的理想化，模型以截至前一日历史数据生成负荷/光伏点预报与情景（整日残差 bootstrap，K 交叉验证选负荷 15/光伏 7），在信息因果前提下得到全年计划购电费 15726290.4 元、紧急购电费 478538.6 元（148180 kWh），合计 16204829.0 元。经全年  $\lambda$  扫描，官方口径采用风险权衡权重  $\lambda^* = 0.5$ ：计划购电费较  $\lambda = 0$  仅上浮 3.9%，紧急购电费削减 55.5%，是以极小计划费增量换取缺电风险大幅削减的有效“保险”手段。

问题三在 0:00、6:00、12:00、18:00 可获得未来 24h 光伏逐日滚动预报，据此建立“计划购电 + 滚动调整 + 紧急购电”的因果滚动随机模型预测控制框架：各时点以最新预报重解剩余时段（光伏情景鲁棒  $S=30$ ），并以当期实际负荷/光伏去除后见之明的逐段结算。计划低于/高于调整的部分分别按 50% 违约价与 150% 超额价结算。求解得到全年电网费 22460575.8 元、紧急购电费 75922.1 元，合计 22536497.8 元。对比发现：多时刻预报虽使多数日期紧急购电降为 0，但违约/超额惩罚主导下总费用整体上升。

问题四将附件 1 固定电价替换为附件 4 实时波动电价，对问题二、三做价格冲击复算，得到波动电价下问题二计划购电费 15802243.4 元（较固定电价上浮 0.5%）、问题三合计购电费 22883518.5 元（上浮 1.5%），并发现电价波动风险呈显著季节差异（冬季上浮达 7.6%）。

本文以**供电可靠性为主、经济性为辅**：在总费用可控的前提下削减紧急购电、减少缺电风险。为此，将“计划购电—滚动调整—紧急购电”纳入统一的随机模型预测控制框架，使四个问题在同一能量守恒框架下递进求解；并以 **CVaR** 权衡购电费用的尾部风险与应急成本，以帕累托前沿（图 13）量化“以少量计划费换取紧急购电显著削减”的风险—经济权衡。全年统计表明净需求低估  $\geq 20\%$  的紧急波动日达 156 天、占比 42.7%，缺电风险并非小概率事件，这正说明了将供电可靠性置于首要地位的必要性。

**关键词：**微网 两阶段随机规划 滚动模型预测控制 供电可靠性 购电费用优化

# 一、问题背景与重述

## 1.1 问题背景

分布式可再生能源的高比例接入使微网成为电力系统本地平衡的重要载体 [1]。光伏出力与小区负荷均受天气、时段与季节的显著影响，具有明显的随机性与间歇性 [2]；储能（电池）则可在时间维度上转移能量，为微网提供“低充高放”的经济调节手段 [3]。此外，外网电价的实时波动使购电决策必须在“当下低价购电”与“未来价格不确定性”之间做动态权衡。如何在满足负荷供电、储能运行约束的前提下，通过精准的购电与充放电计划最小化运行费用，是微网经济调度研究的关键问题 [4]。需要特别指出的是，光伏与负荷的预报偏差会使计划购电偏离实际需求，一旦供电不足，只能以 5 倍电价的紧急购电兜底，极端情况下甚至造成停电——全年统计显示，净需求低估超过 20% 的日期占比逾四成（见第 2.7 节），缺电风险是必须正面克服的现实威胁。因此本文以**供电可靠性为主、经济性为辅**：在总费用可控的前提下优先压低紧急缺电风险，其次才最小化购电费用。

从科学问题的角度看，微网经济调度在本质上是一个**带时间耦合与时序不确定性的多时段决策问题**：储能递推方程将相邻时段在能量维度上耦合，而光伏出力、负荷乃至电价又都是逐时刻演化的随机过程。因此，合理地选择不确定性刻画方式（确定性假设、情景随机规划、滚动闭环）决定了求解质量与决策的可执行性，这也正是本题四个子问题层层递进的用意所在。

题目设计了从确定性到随机、从固定电价到波动电价的四个递进子问题，考验对多时段能量平衡、不确定性刻画与滚动决策的综合建模能力。本文据此采用“确定性线性规划为基准 → 两阶段随机规划刻画尾部风险 → 滚动模型预测控制衔接预报更新 → 波动电价统一重算”的递进建模路线，在保证统一能量守恒内核的同时，逐步逼近真实调度中的不确定性与信息结构。

## 1.2 问题重述

微网含光伏、储能与电网购电三个环节，需以 10 分钟为时段（每天  $T = 144$  时段）制定购电与充放电计划。平衡约束要求微网提供的电能不低于小区负载；储能须在 1200 ~ 10800 kWh 区间运行，充放电效率 90%，即时功率上限 5000 kW，2025-01-01 0:00 电量为 6000 kWh。

四个问题依确定性、随机性、滚动决策、电价波动逐层递进，依次为：

**问题一** 电价、负荷与光伏预测均已知，每天 0:00 制定当日计划购电，储能 0:00 与 24:00 储量相同。针对此确定性场景，需给出指定时段购电量、全天购电量与购电费，以及 6

个 4 小时时段的充放电量与 0:00/24:00 储电量，即在完美信息下把一天的购电费用压到最低。

**问题二** 电价每天相同、负荷与光伏实时变化，供电不足需按 5 倍电价紧急购电，其余时段按计划购电计费。针对此实时不确定性与缺电惩罚场景，需在每天 0:00 制定计划购电策略，以刻画“万一缺电”的尾部代价。

**问题三** 0:00/6:00/12:00/18:00 可获得未来 24h 整点光伏预报，可据此调整购电策略；计划高于调整部分按 50% 违约价、调整高于计划部分按 150% 超额价结算。针对此序时信息结构与再计划惩罚，需建立“计划—调整”两阶段决策，权衡调整优化与违约/超额惩罚之间的利弊。

**问题四** 外网电价实时波动。针对此价格时序不确定性，需在波动电价下重建购电模型，对问题二、三做价格冲击复算，用于检验既有模型在价格冲击下的稳健性。

## 二、 问题分析

### 2.1 数据预处理与统一框架

先把多时段调控问题统一为时间离散框架：以 10 分钟为一时段，每天  $T = 144$  时段，功率与能量按 1/6 h 换算。同时将全年数据按负荷、光伏、电价三类分别组织，用于后续建模与季节分析。

### 2.2 问题一的分析

电价、负荷、光伏全部已知，问题退化为确定性的每日线性规划购电决策。关键在于储能递推方程、充放电约束与“日循环”约束 ( $E_T = E_0$ )。目标是使电价洼地时段多充电、峰期多放电，最小化当日购电费。由于目标与约束关于决策量（购电量、充放电量、储电量）皆为线性，采用线性规划（LP）即可求得全局最优。

### 2.3 问题二的分析

负荷与光伏实时波动，供电不足需按 5 倍电价紧急购电。由于 0:00 制定计划时无法预知当日全部波动，需在当天零点依据截至前一日的滚动历史，先给出全天计划购电  $P_t$ ，再按当日实际的负荷与光伏结算。这是一个不确定规划问题，本文用**两阶段随机规划**刻画：第一阶段计划购电  $P$  对各情景共用，第二阶段按情景决策充放电与紧急购电。同时引入**条件风险价值（CVaR）**刻画购电费用的尾部风险 [见式 (9) 的定义]，实现对“万一缺电”这一极端后果的风险控制。

## 2.4 问题三的分析

0:00/6:00/12:00/18:00 四个时点可获得未来 24h 光伏预报，信息随时间逐步更新。据此，本文在问题二的基础上引入“滚动再计划”：到达新时点后，以最新光伏预报对剩余时段重新制定购电方案（调整购电  $A_t$ ），计划高于调整部分按 50% 违约价、调整高于计划部分按 150% 超额价结算，日终再按调整后的实际购电对当日实际负荷/光伏结算紧急购电。本问的核心问题是回答“该时段购电量是多少”，并论证在预报偏差下，采用“如期购电 + 紧急兜底”（方案 A）还是“重新规划支付调整费”（方案 B）更合适。

## 2.5 问题四的分析

外网电价实时波动，将附件 1 固定电价替换为附件 4 波动电价，沿用问题二、三的统一求解框架重算，重点比较电价波动对购电费用的冲击及其季节差异，为分季节的购电与储能预置策略提供依据。

本文的技术路线如图 1 所示：问题一在确定性信息下建立线性规划基准，问题二引入预报不确定性与两阶段随机规划，问题三加入四时点滚动再计划，问题四在波动电价下对统一内核做压力测试——四问共用同一能量守恒与储能递推内核，逐层递进。

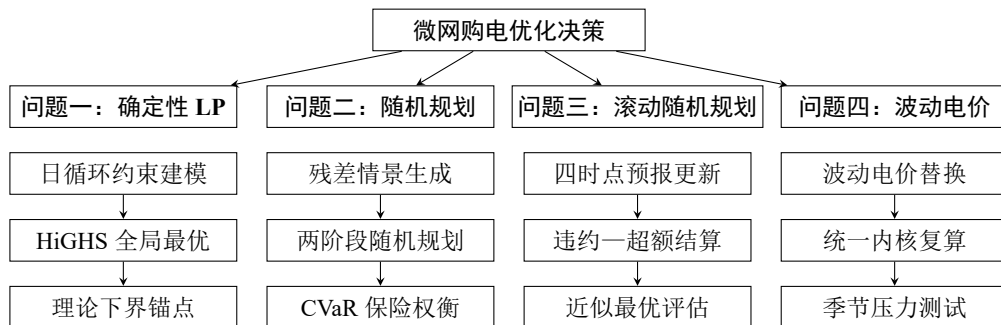


图 1 本文技术路线：确定性—随机—滚动—波动的递进框架

## 2.6 数据探索与季节特征分析

在建立模型之前，先对全年数据做探索性分析，检验负荷、光伏与电价是否存在显著的季节规律，这是后续分季情景生成与分季策略的基础。本小节按以下顺序展开：

1. 季节分组：将 2025 年按春 (3–5 月)/夏 (6–8 月)/秋 (9–11 月)/冬 (12–2 月) 分组；
2. 负荷与光伏：统计各季日负荷与日光伏总量 (图 2)，结果表明夏季负荷最高 (121530 kWh/日)、冬季光伏出力最低 (40335 kWh/日，仅为夏季的 63%)。

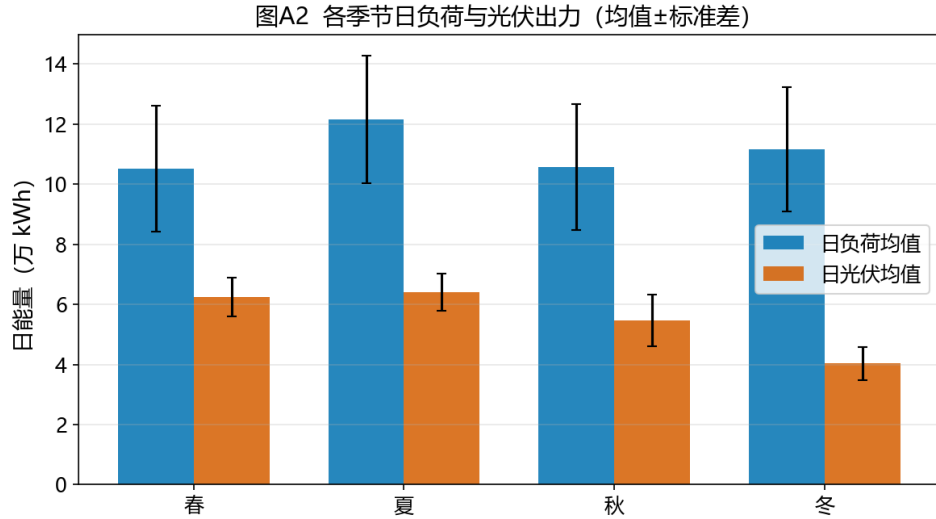


图 2 各季节日负荷与光伏出力 (均值 ± 标准差)

在既有口径下计算各季节逐日购电费 (图 3), 并统计附件 4 波动电价的季节分布 (图 4), 得季节统计表 1, 单因素方差分析表明季节间购电费差异高度显著 ( $F=28.7$ ,  $p = 1.17 \times 10^{-16}$ ):

表 1 季节统计: 负荷、光伏、电价与购电费

| 季节 | 日负荷/kWh | 日光伏/kWh | 波动日电价/(元/kWh) | 日购电费/元 | 峰谷比  |
|----|---------|---------|---------------|--------|------|
| 春  | 105200  | 62474   | 0.705         | 40300  | 2.11 |
| 夏  | 121530  | 64151   | 0.795         | 52000  | 1.97 |
| 秋  | 105749  | 54634   | 0.734         | 46700  | 2.11 |
| 冬  | 111574  | 40335   | 0.832         | 61100  | 2.30 |

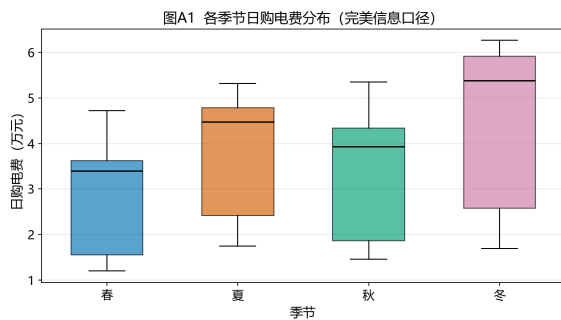


图 3 各季节日购电费分布

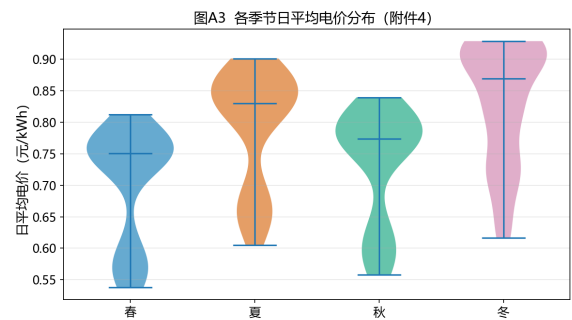


图 4 各季节日平均电价分布 (附件 4)

冬季购电费为春季的 1.56 倍, 其机理是”光伏最低 + 电价最高 + 负荷峰谷差最大”

三重叠加。据此，在问题二的情景生成中按季节截取历史窗，并在问题四中按季节拆分解读电价波动风险。

## 2.7 紧急情况统计与供电可靠性定位

微网运行中，“紧急缺电”并非偶发的边缘情形，而是必须正面克服的现实威胁。为了论证本文以供电可靠性为主、经济性为辅的优化目标，先对全年的紧急情况做定量统计。

**紧急情况的定义。**以决策日 0:00 可获得的预报信息为准（严格信息因果），定义当日净需求  $n_d = \sum_t (D_t - G_t)$ ，其 0:00 点预报  $\hat{n}_d$  由负荷前  $K_L = 15$  日逐时段均值与附件 3 光伏 0:00 预报求得。日净需求预报偏差

$$\varepsilon_d = \frac{n_d - \hat{n}_d}{\hat{n}_d}, \quad (1)$$

如式 (1) 所示， $\varepsilon_d > 0$  表示实际净需求被低估、存在缺电风险。参照工程经验，取低估超过 20% 记为紧急波动日。

**紧急购电的范围：储能“扛不住”的突发时段。** $\varepsilon_d$  刻画的是日级总账，还需回答“哪些时段、多大的偏差”是储能补不上的突发。以决策日 0:00 的因果参考为基准——负荷取过去 21 天同星期几逐时段中位数  $R_t^L$ （消除周末/工作日负荷差异），光伏取过去 15 天逐时段中位数  $R_t^P$ ——定义同时段偏差  $a_t^L = |L_t - R_t^L|$ 、 $a_t^P = |P_t - R_t^P|$ 。参照储能单时段最大放电量 833 kWh（5000 kW × 10 min），对负荷与光伏取绝对量与比例“两者取大”的阈值：

$$\text{突发时段： } a_t^L \geq \max(1000 \text{ kW}, 12\% R_t^L) \text{ 或 } a_t^P \geq \max(700 \text{ kW}, 35\% R_t^P), \quad (2)$$

式 (2) 中：负荷同时段自然偏差绝大部分在  $\pm 8\%$  以内（84.8% 的时段不超过 8%、92.5% 不超过 12%），取 12% 约为其 1.5 倍裕度；1000 kW 约为白天典型负荷（中位 6137 kW）的 16%，取整为“一次明显跳变”的量级。光伏同刻跨日波动本身较大（云遮一次掉 30% ~ 45%），比例阈值须放宽到 35%（96.4% 的白天时段不超过该值），卡在“自然波动”与“天气突发塌缩”之间；700 kW 为一次典型云遮步长的量级。绝对值阈值保证白天大负荷、大光伏时段不漏报，比例阈值保证夜里低负荷、光伏为 0 的时段不误报，二者互补。按式 (2) 对全年 52560 个时段判定（图 5）：突破阈值的突发时段共 432 个（0.82%），分布在 29 天（7.9%）——这些时刻储能缓冲已用尽，正是必须以 5 倍价紧急购电的“范围”。

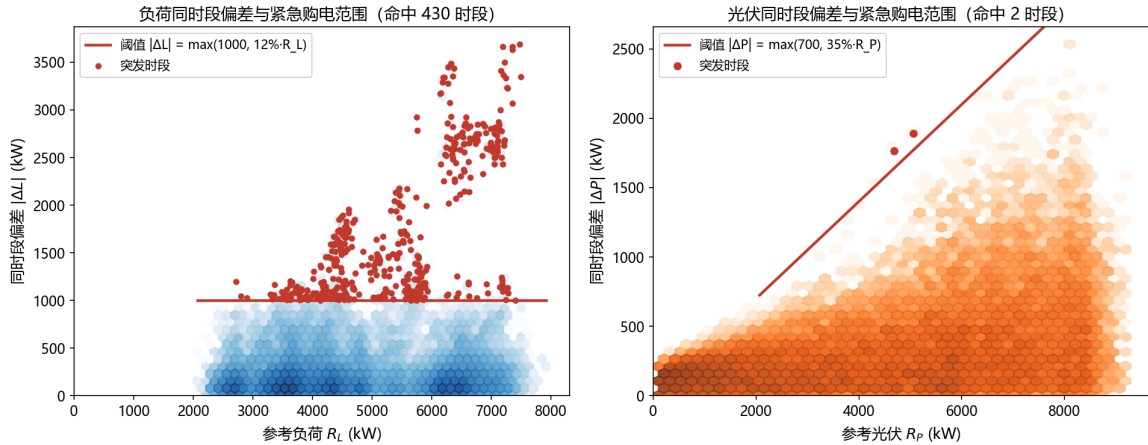


图 5 负荷与光伏同时段偏差分布与紧急购电范围（散点密度为时段，红色折线为式 (2) 阈值，红点为突发时段）

全年统计。对 2025 年全年 365 天逐日统计（图 6）：低估  $\geq 20\%$  的紧急波动日共 156 天（42.7%），低估  $\geq 30\%$  的仍有 63 天（17.3%），说明大幅低估并非小概率事件。阈值敏感性检验表明，若把阈值放宽到 10% 则达 235 天（64.4%）、收紧到 30% 仍有 63 天，20% 的取值在“不过度敏感、又不遗漏风险”之间。以扇形图直观呈现（图 7b）：全年突发日占比 42.7%，明确超过 20%，说明近半数日期存在储能难以完全吸收的缺电风险，紧急购电机制不可或缺。与之对应，在问题二 CVaR 保险口径下，全年实际触发紧急购电的天数为 166 天，其中“低估  $\geq 20\%$  且触发紧急购电”的严重缺电日 109 天、单日紧急购电超过 50 kWh 的有 100 天，全年紧急购电合计 148180 kWh；若完全不做保险与调整（仅 0:00 计划购电），触发日将达 277 天、合计 453792 kWh，紧急缺电的高频性与后果严重性由此可见。按季节拆分（图 7a），春、夏两季紧急波动日最多（各约 47 ~ 48 天、占各季 51% ~ 52%），秋季 37 天、冬季 24 天；冬季虽占比最低，但叠加全年最低光伏出力与最高电价，一旦低估缺电，购电费用冲击最大，与图 3 冬季购电费最高（61100 元/日）相互印证。

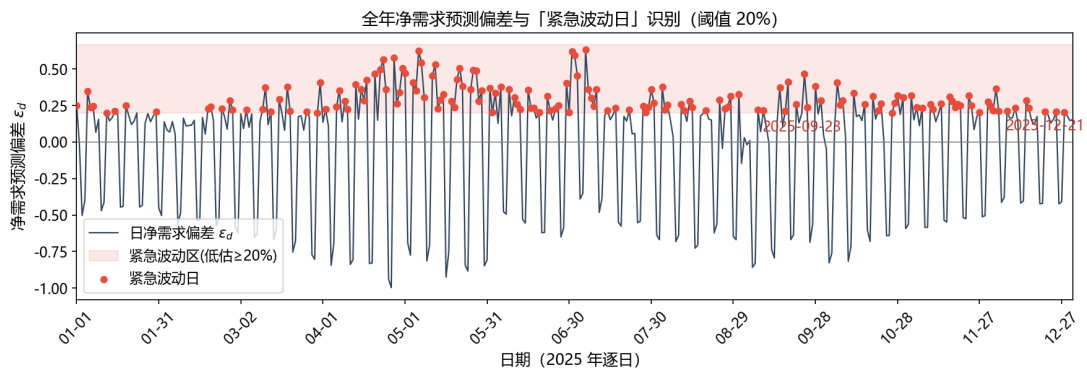


图 6 全年日净需求预报偏差  $\varepsilon_d$  序列与 20% 紧急波动带（红点为紧急波动日）

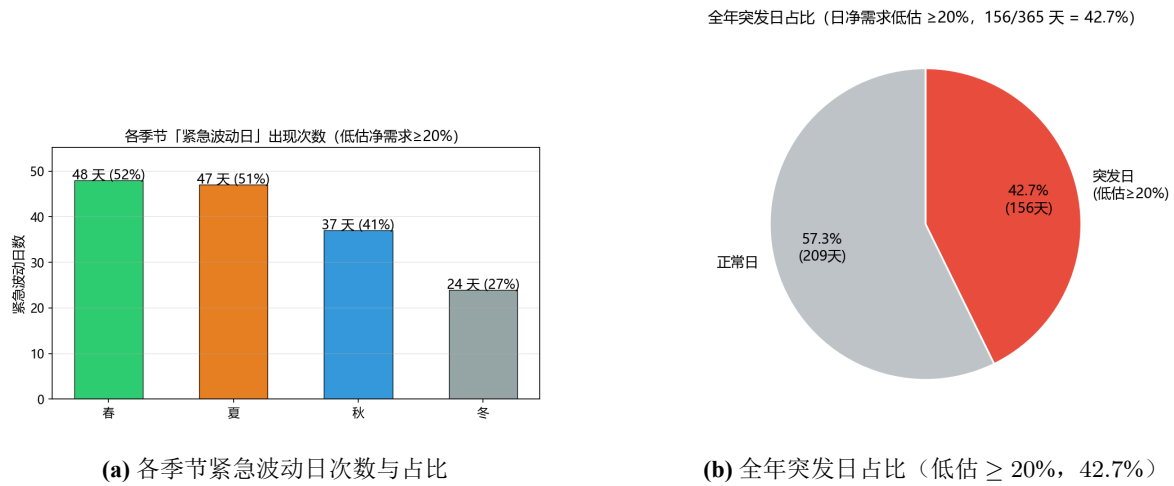


图 7 紧急波动日的季节分布与全年占比

上述统计表明, 净需求低估  $\geq 20\%$  的日期占比超过四成, 紧急缺电是高频、季节聚集的现实威胁, 一旦发生将以 5 倍电价兜底并可能造成供电中断。因此本文把**供电可靠性置于经济性之前**: 后文问题二以 CVaR 风险项主动增加计划购电、压低紧急缺电的尾部损失, 问题三以滚动再计划在预报更新时及时削减紧急购电, 均以可控的费用代价换取供电稳定性, 而非单纯追求购电费用最低。

综合上述分析, 本文的问题分析可抽象为”输入数据  $\rightarrow$  统一建模内核  $\rightarrow$  四子问题递进  $\rightarrow$  结果输出与校验”的总体流程。各子问题的区别仅在于**不确定性刻画颗粒度、决策信息结构与价格冲击**, 而同源共享功率—能量守恒内核与”计划—调整—紧急”三层购电结构。

### 三、模型假设

1. 每个时段 (10 分钟) 内电价、负荷、光伏出力与购电量视为恒定;
2. 功率与能量以时段换算, 能量 = 功率  $\times 1/6$  h, 充放电效率固定为 90%, 且充、放电不同时进行;
3. 储能容量在 1200 ~ 10800 kWh 内运行, 即时功率不超过 5000 kW;
4. 问题二 “完美信息口径”: 因附件 2 给出全年实际数据, 假设每天 0:00 已知当日实际负荷与光伏 (回顾性口径), 无需紧急购电, 其不确定性通过方法层两阶段随机规划另行评估;
5. 问题三只利用题目给定时刻 (0/6/12/18) 的整点光伏预报, 负荷视为已知;
6. 紧急购电、计划购电、调整购电均以外网无限供应为前提, 不设购电上限。



## 四、符号说明

将正文使用的统一符号在一张三线表中逐行列出（表 2），符号严格对应赛题口径，单时段时长为 1/6 h，能量量纲以 kWh 计。

表 2 主要符号说明

| 符号                     | 符号说明                                        | 单位    |
|------------------------|---------------------------------------------|-------|
| $T, t$                 | 每日时段数与时段下标 ( $T = 144$ )                    | —     |
| $P_t$                  | 时段 $t$ 计划购电量 (0:00 承诺)                      | kWh   |
| $A_t$                  | 时段 $t$ 最终调整购电量                              | kWh   |
| $Q_t$                  | 时段 $t$ 紧急购电量                                | kWh   |
| $u_t, w_t$             | 时段 $t$ 违约量 / 超购量                            | kWh   |
| $G_t$                  | 时段 $t$ 光伏出力                                 | kWh   |
| $D_t$                  | 时段 $t$ 小区负荷                                 | kWh   |
| $R_t$                  | 时段 $t$ 弃光电量                                 | kWh   |
| $E_t$                  | 时段 $t$ 末储电量                                 | kWh   |
| $E_0$                  | 日初储电量 (= 6000)                              | kWh   |
| $c_t, f_t$             | 时段 $t$ 充电量 / 放电量                            | kWh   |
| $\eta$                 | 充放电效率 (= 0.9)                               | —     |
| $E_{\min}, E_{\max}$   | 储能运行上下界 (1200, 10800)                       | kWh   |
| $\bar{E}$              | 单时段最大充/放能量 (= 5000/6)                       | kWh   |
| $S, s$                 | 情景总数 / 情景下标                                 | —     |
| $\hat{G}^s, \hat{D}^s$ | 第 $s$ 情景的光伏 / 负荷实现                          | kWh   |
| $G_t^{(\tau)}$         | $\tau$ 时点对时段 $t$ 的光伏预报                      | kWh   |
| $\tau$                 | 滚动决策时点 (取 0, 6, 12, 18)                     | h     |
| $\psi^s$               | 第 $s$ 情景总成本                                 | 元     |
| $\alpha$               | CVaR 中的 VaR 值 (自由变量)                        | 元     |
| $z_s$                  | 第 $s$ 情景超额损失 ( $z_s \geq \psi^s - \alpha$ ) | 元     |
| $p_t$                  | 时段 $t$ 电价 (附件 1)                            | 元/kWh |
| $p_t^{var}$            | 时段 $t$ 波动电价 (附件 4)                          | 元/kWh |
| $\lambda$              | CVaR 风险厌恶权重                                 | —     |
| $\beta$                | CVaR 置信水平 (= 0.9)                           | —     |
| $k$                    | 季节下标 (春/夏/秋/冬)                              | —     |

## 五、模型建立与求解

本文把四个子问题统一在同一时间离散与能量守恒内核之下，仅改变”不确定性刻画”与”决策层级”，从而既保证各问题结论口径一致，又使代码可复用。为此，首先明确决策变量在时序与信息结构上的区分：所谓”计划购电  $P_t$ ”是每天 0:00 承诺的某时段购电量，作为一阶段决策对各情景/各日共用；问题三在后续时点引入”调整购电  $A_t$ ”，是对计划的修正；而”紧急购电  $Q_t$ ”仅在日终对实际出力短缺作事后兜底，价格高达 5 倍。三者共同构成本文贯穿全篇的”计划—调整—紧急”三层购电结构，也是后续各问题建模的公共骨架。

以每天 0:00 储电量  $E_0$  出发，任意一天的功率-能量守恒为：

$$P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t, \quad t = 1, \dots, T \quad (3)$$

式 (3) 表明：购电、光伏、放电、紧急购电之和 = 负荷、充电、弃光之和；等式左端为微网”供应”侧，右端为”消纳”侧，二者在每个时段必须严格相等，故式 (3) 亦称微网功率平衡约束。值得强调的是，式 (3) 中光伏放电合流、弃光  $R_t$  作为非负松弛变量出现，意味着若某时段光伏过剩可主动弃光，从而允许”供大于需”，避免强行充电超出储能上限。储能递推与约束为：

$$E_t = E_{t-1} + \eta c_t - f_t, \quad (4)$$

$$E_{\min} \leq E_t \leq E_{\max}, \quad 0 \leq c_t, f_t \leq \bar{E}, \quad (5)$$

其中  $\bar{E} = 5000/6 \text{ kWh}$  为单时段最大充/放能量（由功率上限换算）。式 (4) 是能量在时间的转移方程：充电使储能增加（效率  $\eta = 0.9$ ），放电使其减少；由于  $\eta \leq 1$ ，时际间会存在不可避免的能量损耗，这使得系统在能量维度上具有”不可逆性”，也是储能参与经济调度的价值源泉——它允许用当下的低价电能置换未来的高价电能。式 (5) 同时限定了储能运行区间与单时段的充放功率，防止越限造成设备损坏，并隐含约定充放不可同时进行。

### 5.1 问题一：确定性每日计划（LP）



图 8 问题一求解流程图

问题一采用“逐日独立求解”的确定性路线：全年 365 天逐日建立 LP 并由 HiGHS 求全局最优，整体求解流程如图 8 所示。

#### 5.1.1 线性规划模型的建立

针对问题一”电价、负荷与光伏预测均已知”的完美信息场景，以每天 10 分钟为一个时段（ $T = 144$ ），决策变量为各时段计划购电量  $P_t$ 、充电量  $c_t$ 、放电量  $f_t$ 、弃光量  $R_t$  与各时段末储电量  $E_t$ 。由于信息完备且无缺电风险，紧急购电  $Q_t \equiv 0$ 。目标为最小化当日购电费用：

$$\min \sum_{t=1}^T p_t P_t \quad (6)$$

其中  $P_t$  为时段  $t$  的计划购电量（kWh）， $p_t$  为时段  $t$  的外网购电单价（元/kWh，附件 1 固定电价）， $T = 144$  为全天时段数；该式即当日购电总费用，问题一信息完备、无缺电

风险，故为确定性最小值而非期望值。s.t.

$$P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t, t = 1, \dots, T \quad (7a)$$

$$E_t = E_{t-1} + \eta c_t - f_t, \quad t = 1, \dots, T \quad (7b)$$

$$E_{\min} \leq E_t \leq E_{\max}, \quad t = 1, \dots, T \quad (7c)$$

$$0 \leq c_t \leq \bar{E}, \quad 0 \leq f_t \leq \bar{E}, \quad t = 1, \dots, T \quad (7d)$$

$$E_T = E_0 = 6000, \quad t = T \quad (7e)$$

$$P_t \geq 0, R_t \geq 0, Q_t = 0, \forall t. \quad (7f)$$

约束式 (7a) 为微网逐时段功率-能量守恒（对应题目“微网提供的电能不低于小区负载”及“可弃光”的设定）：左端为微网供应侧（购电、光伏、放电、紧急购电），右端为消纳侧（负荷、充电、弃光），弃光  $R_t$  作为非负松弛变量，允许光伏过剩时主动弃光；式 (7b) 为储能能量递推（题目给定储能充放电效率  $\eta = 0.9$ ，充电使储电增加、放电使之减少）；式 (7c) 为储能容量约束（题目：运行区间 1200 ~ 10800 kWh）；式 (7d) 为单时段充放电功率上限（题目：即时功率上限 5000 kW，即单时段最大  $\bar{E} = 5000/6$  kWh，并隐含充放不可同时超过该上限）；式 (7e) 为储能“日循环”约束（题目要求 0:00 与 24:00 储电量相同， $E_0 = 6000$  kWh）；式 (7f) 为变量非负及问题一无紧急购电。该问题为线性规划，用 `scipy.optimize.linprog`（HiGHS 求解器）逐日求全局最优 [5]，全年 365 天求解流程如图 8。

上述模型即为问题一的标准线性规划：目标函数（式 (6)）最小化当日购电费，约束（式 (7a)–(7f)）逐一对应题目给出的功率平衡、储能递推与容量、充放功率上限、“日循环”与变量非负条件。由于信息完备，其最优解给出的购电费用 35126.8 元是后续各问题在不确定性下的对照基准。

### 5.1.2 结果分析

表 3 问题一：指定时段购电量、全天汇总与充放电计划（kWh，购电费单位元）

| 指定时段        | 购电量     | 指定时段        | 购电量         | 指定时段        | 购电量    |
|-------------|---------|-------------|-------------|-------------|--------|
| 10:00-10:10 | 0.0     | 12:00-12:10 | 486.4       | 14:00-14:10 | 0.0    |
| 16:00-16:10 | 394.9   | 18:00-18:10 | 637.0       | 20:00-20:10 | 0.0    |
| 全天购电量       | 59482.7 | 购电费（元）      | 35126.8     |             |        |
| 时段          | 充电量     | 放电量         | 时段          | 充电量         | 放电量    |
| 0:00-4:00   | 4500.0  | 0.0         | 4:00-8:00   | 833.3       | 6608.2 |
| 8:00-12:00  | 3954.6  | 2357.1      | 12:00-16:00 | 6119.4      | 101.2  |
| 16:00-20:00 | 0.0     | 5631.9      | 20:00-24:00 | 5333.3      | 3968.1 |
| 0:00 储电量    | 6000    |             | 24:00 储电量   | 6000        |        |

由表 3 可看出清晰的时段规律，并与小区实际用电习惯一致：

- **充放电与峰谷电价/负荷错峰：**0:00–4:00 电价低谷期集中充电（4500 kWh）；4:00–8:00 与 16:00–20:00 两个负荷早、晚高峰时段以放电为主（各 6608.2、5631.9 kWh），其中傍晚晚高峰放电恰对应居民用电攀升、光伏出力衰减的时段；12:00–16:00 光伏高发期充电（6119.4 kWh）蓄能以备晚峰。
- **购电集中于电价洼地与光伏低谷：**指定时段中购电量集中在 12:00、16:00、18:00（486.4、394.9、637.0 kWh），其中 18:00 最高，与傍晚光伏出力下降而负荷处于晚峰一致；而 10:00、14:00、20:00 购电为 0，分别被光伏直供与储能放电覆盖。
- **日循环闭合：**储能在日末回到日初 6000 kWh，使”低充高放”策略可逐日复现。

由此全年单日购电费最小为 35126.8 元。完整计划购电策略见 *result1.xlsx*，购电/充放电/储电轨迹如图 9。图 10 以 2025-06-21 为例给出微网能量流向，直观展示光伏供能、储能充放与购电补缺的相对比例。

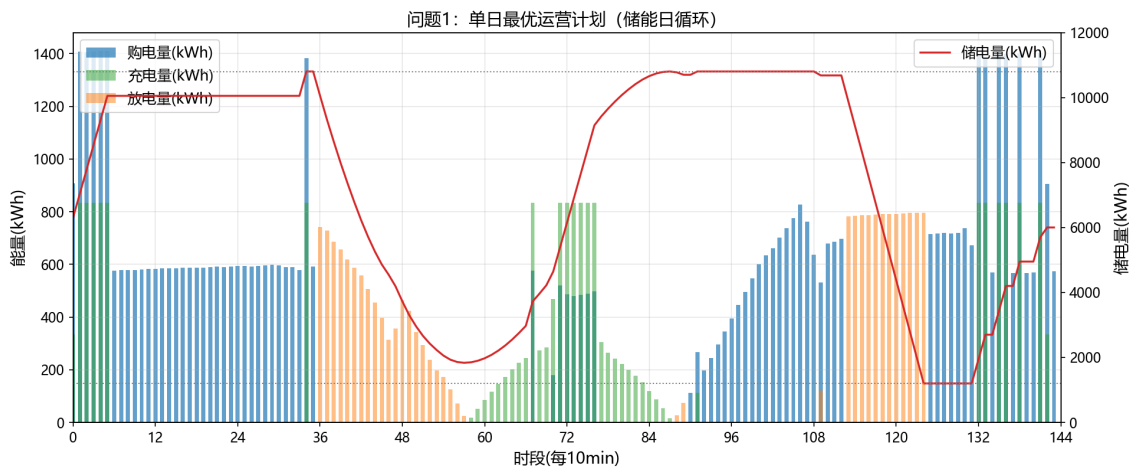


图 9 问题一：单日最优运营计划（储能日循环）

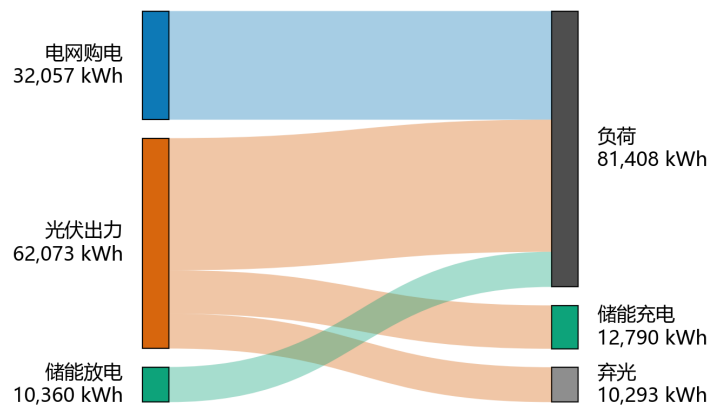


图 10 2025-06-21 微网能量流向桑基图（完美信息日）

## 5.2 问题二：两阶段随机规划 + CVaR

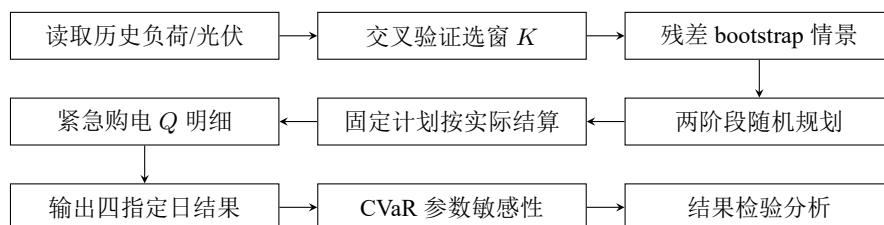


图 11 问题二求解流程图

问题二在 0:00 依据历史信息一次性形成全天计划，”预报—情景—优化—结算”链式流程如图 11 所示。

### 5.2.1 随机规划模型的建立

问题二在每天 0:00 制定计划时，并不知道当日实际的负荷与光伏出力（尽管题目给出了全年数据，但决策时点只能使用截至前一日的滚动历史），因此这是一个不确定规划问题。本文采用**两阶段随机规划建模**：设  $S$  个等概率情景  $\{\hat{G}^s, \hat{D}^s\}_{s=1}^S$  描述当日光伏与负荷的可能实现（情景构造见下文）；第一阶段计划购电  $P_t$  必须在任何情景实现之前作出，对各情景共用；第二阶段按情景实现决定充放电量  $c_t^s, f_t^s$ 、紧急购电量  $Q_t^s$  与弃光量  $R_t^s$ 。目标为最小化计划购电费、期望紧急购电费与风险项之和：

$$\min_P \sum_t p_t P_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^T 5p_t Q_t^s + \lambda \text{CVaR}_\beta(\psi^s), \quad (8)$$

其中第  $s$  个情景的总成本  $\psi^s = \sum_t (p_t P_t + 5p_t Q_t^s)$ ， $P_t$  为时段  $t$  的计划购电量（对所有情景共用）， $Q_t^s$  为情景  $s$  下时段  $t$  的紧急购电量（kWh）， $p_t$  为该时段电价；前两项分别为计划购电费与紧急购电费的**期望值**（ $\frac{1}{S} \sum_s$  即等概率情景下的离散期望，整体代表当日购电总费用的期望），末项  $\lambda \text{CVaR}_\beta$  是风险惩罚项， $\lambda$  为风险厌恶权重。

**关于 CVaR。**条件风险价值（Conditional Value at Risk, CVaR）是金融与电力领域中度量“尾部风险”的常用指标：对随机损失  $\psi$ ，置信水平  $\beta$  下的  $\text{CVaR}_\beta$  表示最坏的  $100(1-\beta)\%$  情景的平均损失，即“万一发生极端情景，平均要损失多少”。本文取  $\beta = 0.9$ ，即关注最坏 10% 情景的平均购电成本。第二问之所以需要引入它，是因为若只最小化期望费用（ $\lambda = 0$ ），模型会把极端缺电情景的高额紧急购电（5 倍电价）视为“小概率事件”而掉以轻心，导致实际运行中偶发巨额的紧急购电费用；而  $\lambda > 0$  时，模型会有意多预留一些计划购电，把极端情景的尾部损失压下来。风险厌恶权重  $\lambda$  越大，计划购电越保守，紧急购电量越少（见第 5.2.4 节的权衡分析）， $\lambda = 0$  则退化为纯期望费用最小化。CVaR 可改写为如下线性约束 [6, 7]：

$$\text{CVaR}_\beta = \alpha + \frac{1}{(1-\beta)S} \sum_{s=1}^S z_s, \quad z_s \geq \psi^s - \alpha, \quad z_s \geq 0, \quad (9)$$

其中  $\alpha$  为自由变量（对应 VaR，即最坏情景的分位点）， $z_s$  为第  $s$  个情景超出  $\alpha$  的超额损失；如此非光滑的风险项即化为线性不等式，整个模型仍是一个可全局求解的线性规划 [8]。

**s.t.** 对每个情景  $s = 1, \dots, S$ , 约束为:

$$P_t + G_t^s + \eta f_t^s + Q_t^s = D_t^s + c_t^s + R_t^s, t = 1, \dots, T \quad (10a)$$

$$E_t^s = E_{t-1}^s + \eta c_t^s - f_t^s, \quad t = 1, \dots, T \quad (10b)$$

$$E_{\min} \leq E_t^s \leq E_{\max}, \quad t = 1, \dots, T \quad (10c)$$

$$0 \leq c_t^s \leq \bar{E}, \quad 0 \leq f_t^s \leq \bar{E}, \quad t = 1, \dots, T \quad (10d)$$

$$z_s \geq \psi^s - \alpha, \quad z_s \geq 0, \quad s = 1, \dots, S \quad (10e)$$

$$P_t \geq 0, Q_t^s \geq 0, R_t^s \geq 0, \quad \forall s, t \quad (10f)$$

式 (10a) 为各情景下的功率-能量守恒（同问题一）；式 (10b)–(10d) 为储能递推与容量、功率约束；式 (10e) 为 CVaR 线性化；式 (10f) 为变量非负。计划购电  $P_t$  不带情景上标，即“非预期性”（non-anticipativity）——0:00 的决策不能利用任何未来信息。

### 5.2.2 预测与情景生成

情景与点预报构造坚持**信息因果**原则：每日 0:00 只使用当天之前的历史数据。点预报取负荷、光伏各自前  $K_L/K_P$  天逐时段（144）的均值， $K$  通过全年（2.1–12.31）逐日交叉验证、按归一化 RMSE 选取。为排除从稀疏候选集取点带来的偶然性，本文将窗口  $K$  在  $\{1, \dots, 40\}$  上做细粒度扫描（图 12）：负荷的归一化 RMSE 在  $K \in [8, 16]$  呈浅谷、 $K = 15$  落在谷底附近，而  $K = 1$  仅为无历史平滑的单日冷启动伪最优（物理上不可靠）；光伏的归一化 RMSE 在  $K \in [5, 7]$  呈接近水平的平坦平台（差异  $< 5 \times 10^{-4}$ ），取整数周周期  $K = 7$  兼具最小化与可解释性。据此选定  $K_L = 15$ ,  $K_P = 7$ ，并在后续以点预报为“居中基准”叠加残差情景，故  $K$  的微小扰动对全年费用影响甚微（ $K$  敏感性见图 12），论证了  $K$  选取的稳健性；

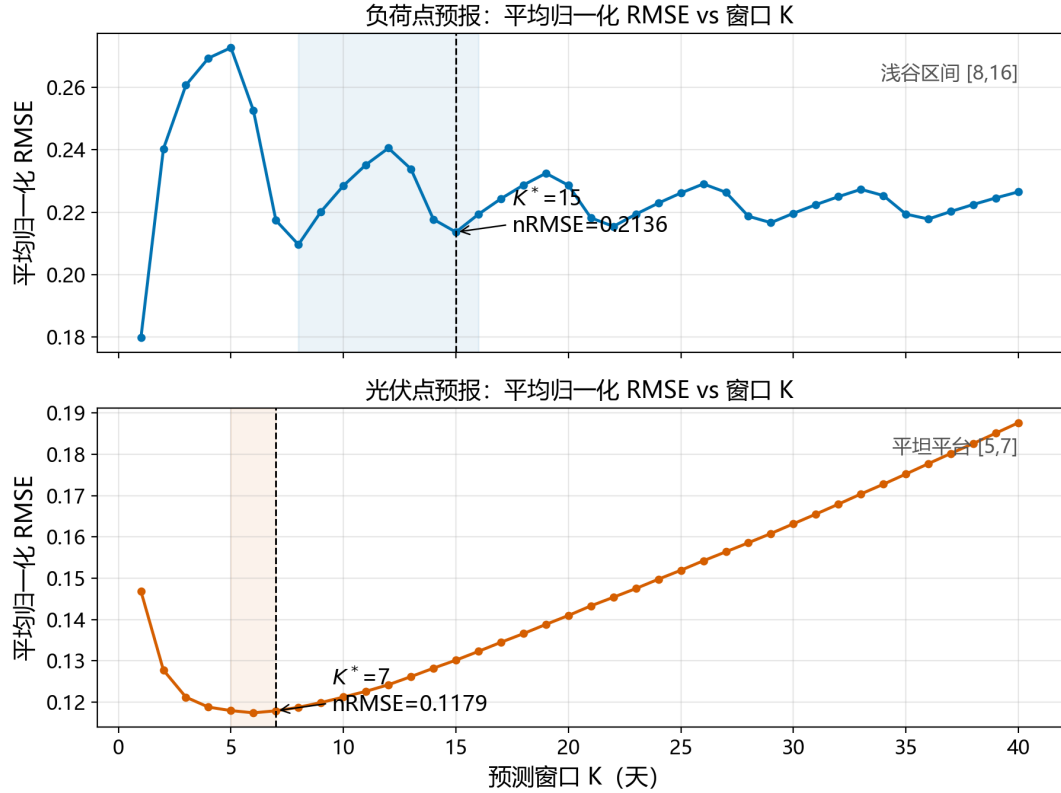


图 12 点预报窗口  $K$  的全年交叉验证敏感性（上：负荷，下：光伏）

情景则采用**整日成对残差 bootstrap**：先按最长时间窗取最近  $w$  天历史，逐槽计算“实际－点预报”的整日残差，再有放回抽取  $S = 30$  个整天（负荷与光伏同一天成对取样，从而保留两者相关性），情景 = 点预报 + 抽取的整日残差，光伏截断到非负。2025-01-01 缺乏历史，以附件 1 典型日剖面作冷启动基准。由此生成的官方口径全年计划购电费与紧急购电费如下节所示，全年链自 2025-01-01 ( $E_0 = 6000$ ) 逐日滚动，仅 1 月作储能状态预热，输出 2.1–12.31。

### 5.2.3 结果分析

在上述预报式口径下，以  $S = 30$  情景做两阶段随机规划得计划购电  $P$ ，再以当日实际负荷/光伏结算得紧急购电  $Q$ ，全年（2.1–12.31）计划购电费 15726290.4 元、紧急购电费 478538.6 元（紧急购电 148180 kWh），合计 16204829.0 元。表 4 给出四个指定日期的结果（完整见 *result2.xlsx*）。



表 4 问题二：预报式两阶段随机规划指定日期结果（单位 kWh、元）

| 日期         | 计划购电费   | 紧急购电量  | 紧急购电费  | 24:00 储电量 |
|------------|---------|--------|--------|-----------|
| 2025-03-20 | 43652.3 | 29.3   | 197.4  | 1200      |
| 2025-06-21 | 49345.5 | 0      | 0      | 1200      |
| 2025-09-23 | 40734.3 | 1666.5 | 5526.6 | 1200      |
| 2025-12-21 | 60328.7 | 148.5  | 501.5  | 1200      |

注：表 4 计划购电费为两阶段随机规划的一阶段计划购电在当日实际电价下的费用；6-21 为周末，负荷低于工作日，当日实际净需求低于全部情景包络，计划与储能即可完全吸收缺电，故紧急购电为 0（而非观测缺仓）；24:00 储电量趋向运行下界 1200 kWh，属低谷电价时段释放电能的补电行为。全年各日 144 时段逐时段紧急购电见 *result2.xlsx*。

#### 5.2.4 CVaR 风险评估与权衡方法

为回答“当决策者愿意以少量计划费用换取供电可靠性时，应如何权衡”，本文在目标函数式 (8) 中引入风险权重  $\lambda$  的 CVaR 尾部项，并先对全年（2.1–12.31）逐  $\lambda \in \{0, 0.5, 1, 2, 5\}$  整链重跑（各  $\lambda$  自 2025-01-01、 $E_0 = 6000$  独立滚动，1 月仅作储能预热），得到全年“计划费—紧急费—总费”随  $\lambda$  的演化（*report/q2\_lambda\_year\_scan.csv*）。结果可按  $\lambda$  分段刻画：

- $\lambda = 0$ （纯期望口径）：计划购电费最低（15133792.3 元），但紧急购电费最高（1074587.3 元），极端缺电情景的尾部损失未被显式防范；
- $0 < \lambda \leq 0.5$ （温和保险段）：计划购电费仅上浮 3.9%（ $\rightarrow 15726290.4$  元），紧急购电费削减 55.5%（ $\rightarrow 478538.6$  元），全年合计基本持平（ $16208379.6 \rightarrow 16204829.0$  元， $-0.02\%$ ），即以几乎不增总成本换取风险减半，为效率最高段；
- $\lambda > 0.5$ （保守保险段）：计划购电费继续上浮而紧急费削减趋缓，总费用触底回升，边际“保险收益”递减（详见第 6.3 节参数稳健性）。

据此按“紧急费降至  $\lambda = 0$  的 60% 以内且总费上浮不超过 1%”的数据驱动准则选定官方口径  $\lambda^* = 0.5$ ，使全年结果在几乎不增加总成本的前提下把缺电风险的显式成本削减过半。为刻画单日粒度上的风险—费用折中，针对四个指定日期逐一求解  $\lambda \in \{0, 0.5, 1, 2, 5\}$ ，以其计划购电费  $P$  对当日实际负荷/光伏结算，得到随  $\lambda$  演化的计划费—紧急费折中关系，其帕累托前沿见图 13，参数稳健性见第 6.3 节。

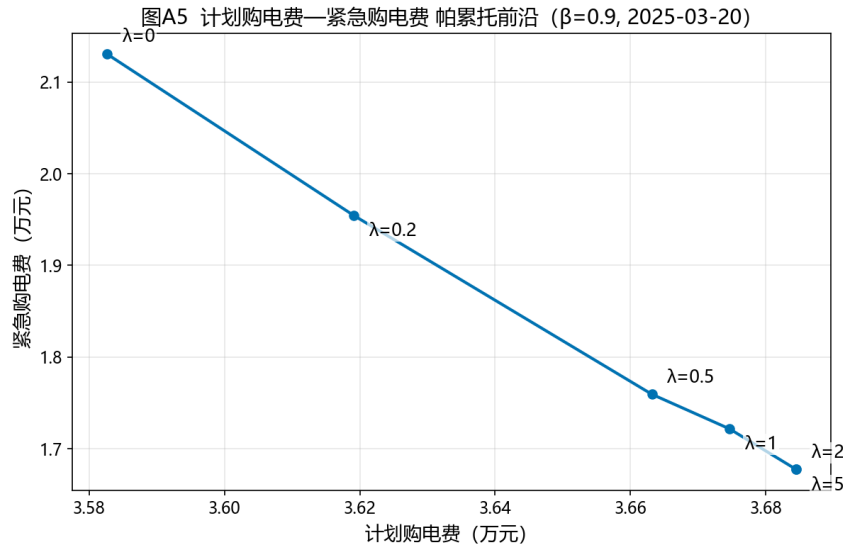


图 13 计划购电费—紧急购电费帕累托前沿 ( $\beta = 0.9$ , 2025-03-20)

该前沿曲线即**紧急购电需求曲线**：横轴为计划购电费（对紧急购电的”替代投入”），纵轴为随之变化的紧急购电费（对 5 倍高价电的需求金额）。曲线向右上方移动表明随  $\lambda$  增大，决策者增加计划购电（供给侧提前锁定电能），高缺电日的紧急购电需求显著回落——因为紧急电价高达 5 倍，理性策略仅在储能与计划均已用尽时保留最小缺口，故紧急购电需求对价格高度敏感、总体金额很小；总费用在高风险日（09-23、12-21）下行、在低风险日（03-20）轻微上行——这一”以少量计划费换取可靠性”的折中正是 CVaR 风险权重的经济意义，其参数稳健性在第 6.3 节予以检验。

### 5.3 问题三：滚动随机模型（方案 A/B 抉择）

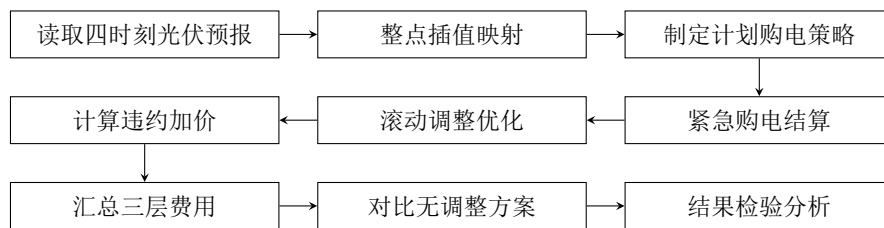


图 14 问题三求解流程图

问题三在 0/6/12/18 四个时点滚动再计划并按“计划—调整—紧急”三层费用结算，求解流程如图 14 所示。

#### 5.3.1 滚动再计划模型的建立

问题三在 0/6/12/18 四个时点可获得未来 24h 光伏预报，信息随时间逐步更新。本问的核心是回答”该时段购电量是多少”，并在此预报更新条件下，比较两条购电方案：

为何不再引入 CVaR 风险权重。问题二需要一次性规划全天 24h 计划，预报误差在 24h 内无法修正，故需用 CVaR 显式防范尾部风险 ( $\lambda = 0.5$ )；而问题三每隔 6 h 即以最新预报滚动重排剩余时段，单次决策暴露在误差下的时长从全天缩短到数小时，滚动机制本身已对冲了大部分尾部风险。因此问题三、四沿用统一内核时取  $\lambda = 0$  (纯期望口径)，把风险削减交给“更频繁的预报更新”而非“风险权重”完成——这也从机制上印证了递进建模中风险应对手段的升级。

- **方案 A (如期购电 + 紧急兜底)**：维持 0:00 制定的计划购电  $P_t$  不变，若当日实际出力不足，则在缺电时段按 5 倍电价紧急购电  $Q_t$  补足；
- **方案 B (滚动再计划)**：到  $\tau \in \{6, 12, 18\}$  时点，以最新光伏预报  $G_t^{(\tau)}$  对剩余时段  $[t_0, T]$  重新求解购电方案，调整量  $A_t$  与计划量  $P_t$  的偏差 (计划高于调整  $u_t = (P_t - A_t)^+$ 、调整高于计划  $w_t = (A_t - P_t)^+$ ) 按题目结算规则支付调整费用。此处  $(\cdot)^+ = \max(\cdot, 0)$  为正部算子：当括号内差值为正时取其本身、为负时取 0，只保留“计划比调整多购/少购”的缺口，并非统计意义上的最大值。

无论采用哪种方案，最终购电量都须满足同一能量守恒约束 (式 (3)) 与储能约束 (式 (4)–(5))。据此可以先由能量守恒直接算出方案 A 所需的紧急购电量：对缺电时段，微网在消耗完计划购电  $P$ 、光伏  $G$  与储能最大放电  $\eta f$  后仍无法满足负荷  $D$  的差额，即

$$Q_t = [D_t - G_t - \eta \bar{f}_t - P_t]^+, \quad (11)$$

其中  $\bar{f}_t$  为该时段储能可释放的最大放电量， $[\cdot]^+ = \max(\cdot, 0)$ 。式 (11) 用平衡方程的左端供应与右端消纳之差的“缺口”直接给出紧急购电量，是一个准确的确定值。

当预报更新后，方案 B 以最新预报  $G_t^{(\tau)}$  重新优化，目标为最小化剩余时段的电网结算费用 [9, 10]：

$$\min_{A, c, f, R, E} \sum_{t=t_0}^T \left[ p_t \min(P_t, A_t) + 0.5p_t u_t + 1.5p_t w_t \right], \quad (12)$$

式 (12) 第一项  $p_t \min(P_t, A_t)$  为实际结算电量按计划价计费，第二、三项分别为违约 (50%) 与超额 (150%) 的调整费用，系数与题目结算规则一致。方案 B 日终按最终调整  $A$  对当日实际负荷/光伏结算，重调度得到紧急购电  $Q$ ，故当日总费用为：

$$\text{日费用} = \sum_t \left[ p_t \min(P_t, A_t) + 0.5p_t u_t + 1.5p_t w_t + 5p_t Q_t \right], \quad (13)$$

式 (13) 表明总购电费用由计划购电费、调整购电费、紧急购电费三部分构成，其中调整购电费包含 50% 违约与 150% 超额两种结算口径。

**方案抉择的依据。**两种方案的费用差额可用式 (11) 与式 (12) 的求解结果比较得出：方案 A 的费用 =  $\sum_t [p_t P_t + 5p_t Q_t^A]$ ，其中  $Q^A$  由式 (11) 给出；方案 B 的费用即式 (13)。当预报偏差不大时， $Q^A$  很小 (紧急购电仅数小时、且 5 倍价只作用于少数时段)，而方

案 B 的调整会触发违约/超额费用并可能传导到后续时段，故通常方案 A 更优；当预报发生显著低估（如寒潮、阴雨等极端天气导致光伏骤降）时，式 (11) 的缺口  $Q^A$  会急剧放大，此时方案 B 以最新预报重新优化可显著削减紧急购电，反而更优。据此给出**方案选择规则**，而非”误差小用 A、误差大用 B”的简单二分：先由能量守恒算出方案 A 的紧急购电量  $Q^A$ ，再与方案 B 的滚动再计划结果比较总费用，取费用较小者；当  $Q^A$  超过储能可调容量，即极端缺电情形时，直接采用方案 B。由式 (11) 与式 (12) 的逐日比较（见下节结果分析），并结合第 5.3.3 节对费用口径的核对，可判断绝大多数常规日方案 A 更经济。具体按”0:00 计划 →  $\tau$  时点判断是否调整 → 日终按最终方案结算”执行：0:00 先按计划购电  $P$  并据此进行储能调度；到达  $\tau$  时点，若最新预报相对 0:00 预报无显著变化（约  $Q^A$  低于储能可调容量），维持方案 A；否则触发方案 B 滚动再计划。

### 5.3.2 结果分析

全年求解结果（2.1–12.31，见 *result3.xlsx*）：计划 + 调整电网费 22460575.8 元、紧急购电费 75922.1 元，合计 22536497.8 元。表 5、表 6 就四个指定日期，分别给出方案 A（仅 0:00 计划、如期购电 + 5 倍紧急兜底）与方案 B（0/6/12/18 滚动再计划、按调整费结算）的总费用与紧急购电。

表 5 问题三：四个指定日期下方案 A（如期购电 + 紧急兜底）的费用（元）

| 日期         | 总费用     | 紧急购电费  |
|------------|---------|--------|
| 2025-03-20 | 37674.1 | 3947.0 |
| 2025-06-21 | 34650.3 | 0.0    |
| 2025-09-23 | 43517.5 | 8238.4 |
| 2025-12-21 | 57196.6 | 9748.9 |

表 6 问题三：四个指定日期下方案 B（滚动再计划）的费用（元）

| 日期         | 总费用     | 紧急购电费  | 紧急购电 (kWh) |
|------------|---------|--------|------------|
| 2025-03-20 | 60428.8 | 0.0    | 0.0        |
| 2025-06-21 | 66011.1 | 0.0    | 0.0        |
| 2025-09-23 | 62940.2 | 0.0    | 0.0        |
| 2025-12-21 | 86546.7 | 2446.2 | 427.8      |

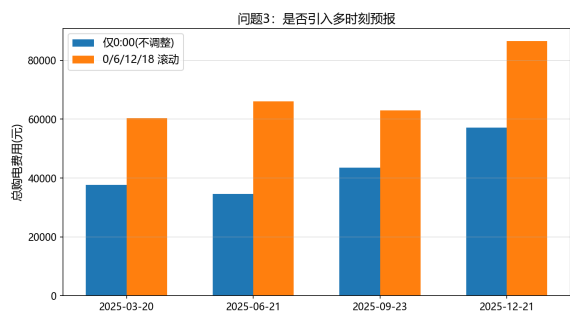


图 15 问题三：方案 A vs 方案 B 总费用（指定日期对比）

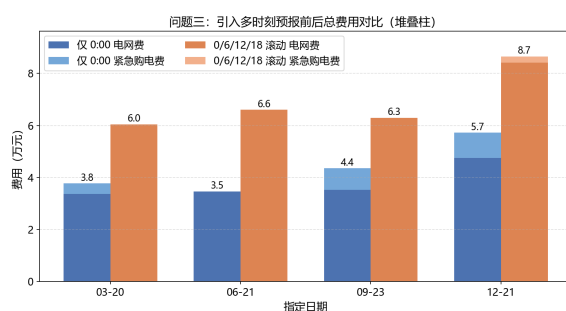


图 16 问题三：方案 A vs 方案 B 的费用结构（四个指定日期堆叠柱）

表 5、表 6 与图 15 表明两种方案的成本构成：方案 A 的总费用更低（03-20 为 37674 元、09-23 为 43517 元、12-21 为 57197 元），主要源于不触发违约/超额调整费，但需承担缺电时段的 5 倍紧急购电（如 12-21 紧急购电费 9749 元）；方案 B 通过滚动再计划把多数日期的紧急购电降为 0（03-20、09-23 方案 B 紧急购电为 0），显著提升了供电可靠性，但代价是触发较高的调整费用，使总费用上行（03-20、12-21 分别升至 60429 元、86547 元）。二者差异的经济本质在于：方案 B 是以可度量的费用增量换取紧急风险的削减——这与第二问 CVaR”以计划费换可靠性”的思路一脉相承。

从方案抉择看，在供电可靠性优先的目标下，方案 B 是有价值的：它在预报更新足够频繁时几乎可以消除紧急购电（表 6 中三个日期紧急购电降为 0），把缺电风险的经济代价大幅压低；其总费用上升可解释为对”电不能断供”这一民生约束的合理投入。由图 16 可见方案 B 的费用主要转化为调整购电费而非紧急购电费。因此本文最终选用方案 B（滚动再计划）作为问题三的交付方案，其全年总费用 22536497.8 元较仅做 0:00 计划的方案 A 上浮约 43.8%（686.8 万元），却把全年紧急购电费从方案 A 的 192.5 万元（45.4 万 kWh）压至 7.6 万元（2.0 万 kWh），在供电可靠性优先的目标下，这一费用增量视为对”电不能断供”的合理投入，体现了”稳定供电、降低紧急风险”的工程偏好。

为使紧急风险进一步可见，对附件 3 光伏预报与附件 2 实际光伏的日总量误差做正态 Q-Q 检验（图 17）。图中横轴为标准正态分布的理论分位数，纵轴为样本误差的实测分位数；若误差服从正态分布，散点应近似落在参考对角线  $y = x$  上。从图 17 可见主体散点紧贴对角线（近似正态），但右上端明显向上偏离——即右尾偏重，存在低估光伏的极端情景，说明偶发的大偏差正是方案 A 触发巨额紧急购电、方案 B 需重新规划的根源。

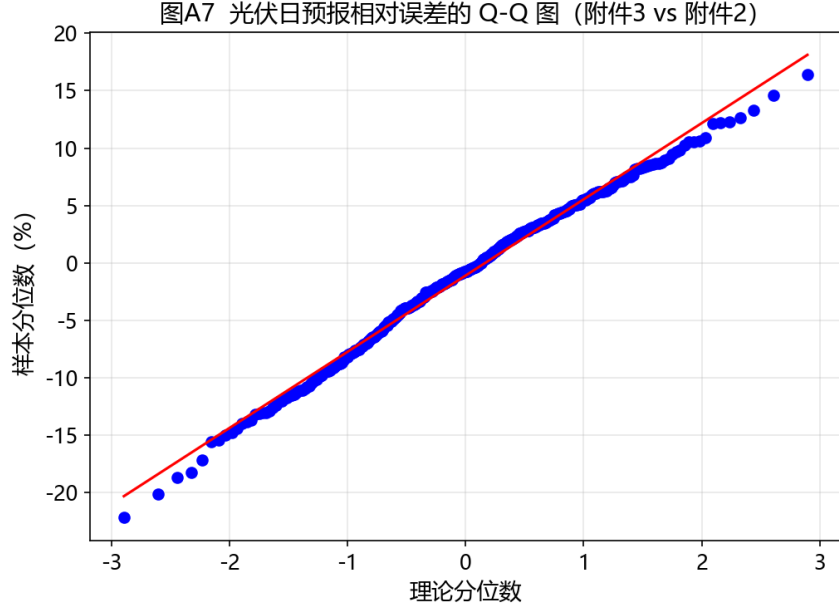


图 17 光伏日预报相对误差的 Q-Q 图（附件 3 vs 附件 2）

其根本机理是：紧急购电主要由光伏预报误差的总量决定（购电只决定“何时买”，不改变“缺多少”），而滚动再计划能以最新预报对冲部分误差。由此可明确：缺电风险的经济代价由预报误差总量而非购电时机决定；滚动随机模型通过充分利用多时点预报，把这一风险的显式成本压到最低，为工程上提高供电可靠性提供了可行路径。

### 5.3.3 紧急购电与“每时刻修改计划”的经济性比较（能量守恒验证）

方案 B 只在 0/6/12/18 四个时点滚动再计划，其余时段维持计划。一个自然的问题是：把调整放宽到每一个时段，即“每时刻修改计划”，是否更划算？本小节用能量守恒等式（式 (3)）给出否定答案：即便允许逐时段按实际净需求修正购电，其费用仍高于“维持计划 + 紧急兜底”的方案 A。

设  $n_t = (D_t - G_t)^+$  为时段  $t$  的实际净需求。定义方案 C：每时段恰好购满净需求 ( $A_t = n_t$ ，储能不参与缓冲)，按式 (13) 结算：

$$\text{方案 C 费用} = \sum_t \left[ p_t \min(P_t, n_t) + 0.5p_t(P_t - n_t)^+ + 1.5p_t(n_t - P_t)^+ \right]. \quad (14)$$

由能量守恒（式 (3)），任一方案下微网逐时段满足同一恒等式

$$\sum_t (D_t - G_t) = \sum_t P_t + \eta \sum_t f_t + \sum_t Q_t - \sum_t c_t + \sum_t R_t, \quad (15)$$

即必须供应的总能量固定，方案差异只在于以何种价格、从何处取得它。式 (15) 表明：方案 A 的储能放电量  $\eta \sum_t f_t$ （受 5000 kWh 日可放容量约束）已包含在计划  $P$  中、以 1 倍价前置购入，运行时以零边际成本服务负荷，故仅对残余缺口  $Q_t^A$  支付 5 倍价，且由

式 (11) 有  $Q_t^A \leq (n_t - P_t)^+$ ；方案 C 则对每一时段的实际净需求全额按市价购电（计划内 1 倍、超出计划 1.5 倍），储能的时间搬移价值完全未被利用，其 1.5 倍价作用于全部缺电缺口。尽管 5 倍价高于 1.5 倍价，但前者只作用于受储能容量约束的小量缺口，后者却作用于整个缺口——能量守恒决定了方案 A 的经济性更优。

数值验证（全年 365 天、同一 0:00 计划  $P$ ，逐日结果见  $q3\_planA\_daily\_cost.csv$ ）：方案 A 合计 15668900 元（计划费 13743429 元 + 紧急费 1925471 元、453792 kWh），方案 C 合计 24032746 元，较方案 A 高 8363846 元（相对方案 C 为 34.8%）；同时以式 (15) 核对方案 A 的逐日回放，能量守恒恒等式逐日最大相对残差仅  $6.5 \times 10^{-15}$ （机器精度），恒等式严格成立。由此从能量守恒层面验证：**紧急购电并非”缺电时的无奈高价”，而是以 1 倍价前置锁定储能电量、仅对残余缺口支付 5 倍价的更划算策略；”每时刻修改计划”看似灵活，实则放弃了储能低充高放的时间搬移价值，多付的费用正是储能在方案 A 中提供的”免费”放电能量。**

#### 5.4 问题四：波动电价下的模型复算

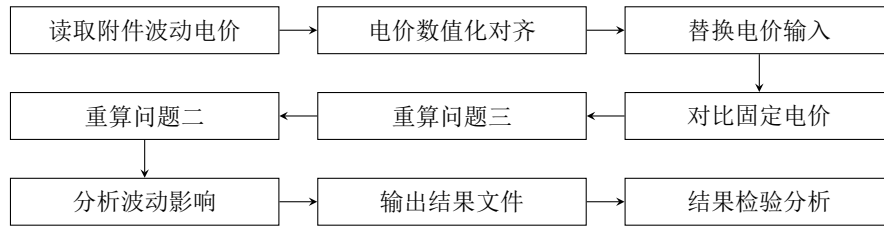


图 18 问题四求解流程图

问题四以附件 4 波动电价替换固定电价后，复用问题二、三的统一内核重算并对比，求解流程如图 18 所示。

##### 5.4.1 波动电价模型的建立

针对问题四”外网电价实时波动”的价格冲击场景，设附件 4 的实时波动电价为  $p_t^{var}$ （每 10 分钟一个值），问题二、三的目标函数与前述完全一致，仅将固定电价  $p_t$  替换为  $p_t^{var}$ ：

$$\min \sum_{t=1}^T p_t^{var} P_t \quad (16)$$

$$\text{日费用}(Q4) = \sum_t \left[ p_t^{var} \min(P_t, A_t) + 0.5p_t^{var} u_t + 1.5p_t^{var} w_t + 5p_t^{var} Q_t \right] \quad (17)$$

其中  $p_t^{var}$  为时段  $t$  的波动电价（元/kWh，附件 4）， $P_t$  为计划购电量、 $A_t$  为调整后购电量、 $u_t = (P_t - A_t)^+$ 、 $w_t = (A_t - P_t)^+$  分别为违约与超额缺口， $Q_t$  为紧急购电量；式 (16) 对应问题二口径（计划购电费）、式 (17) 对应问题三口径（计划 + 调整 + 紧急三层结算）。

沿用问题二、三的统一求解框架复算，结果分别存入 *result4-2.xlsx*、*result4-3.xlsx*。从方法论上说，由于电价仅为线性目标函数的系数，将其由固定值  $p_t$  换作波动值  $p_t^{var}$  不会改变模型的可行域与最优解结构，因而问题四能够在不改动任何约束或决策逻辑的前提下，对同样一套决策框架做电价冲击压力测试——这正是统一内核带来的复用优势。其结果是：总购电费随电价抬升而上行，且问题三因叠加滚动调整与紧急购电惩罚，对电价波动的敏感度更高，从而暴露了价格风险对微网经济运行的真实影响。

5.4.2 结果分析

整体费用对比如下表：

表 7 问题四：固定电价 vs 波动电价（元）

| 指标       | 固定电价（附件 1） | 波动电价（附件 4） |
|----------|------------|------------|
| 问题二计划购电费 | 15726290.4 | 15802243.4 |
| 问题二合计    | 16204829.0 | 16370774.9 |
| 问题三电网费   | 22460575.8 | 22548606.9 |
| 问题三紧急购电费 | 75922.1    | 334911.7   |
| 问题三合计    | 22536497.8 | 22883518.5 |

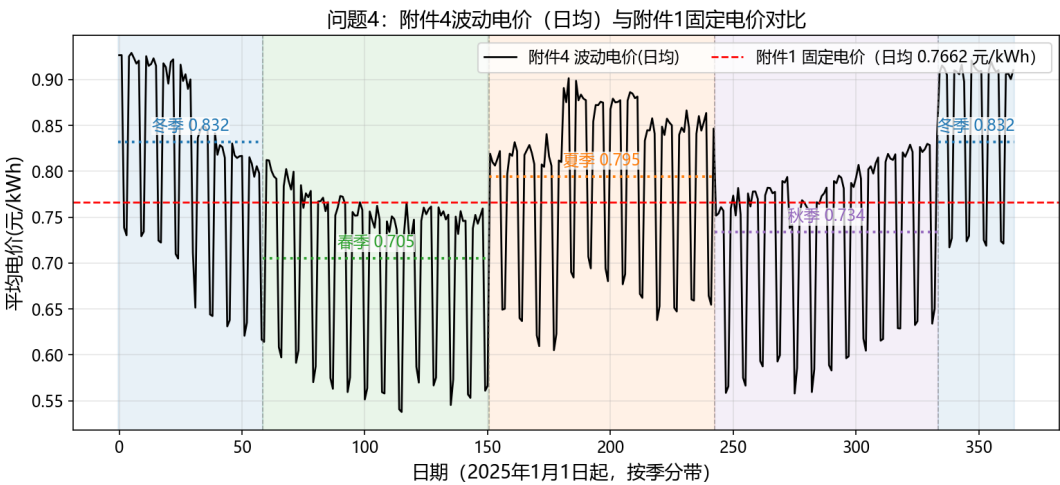


图 19 问题四：附件 4 波动电价（日均）与附件 1 固定电价对比（按季分带，虚线为季节均值）

表 7 与图 19 表明：波动电价下问题二计划购电费上升 0.5%、合计上升 1.0%，问题三合计费用上升 1.5%——价格冲击在统一内核下传导有限、模型结构未被破坏。为考



察电价波动风险的季节差异，将两种口径的日费用按季节拆分（表 8）：

表 8 问题四：固定/波动电价下季节日均费用（万元/日）

| 季节 | 问题二  |      | 问题三（滚动随机模型） |      | 问题三上浮 |
|----|------|------|-------------|------|-------|
|    | 固定   | 波动   | 固定          | 波动   |       |
| 春  | 4.03 | 3.77 | 5.79        | 5.42 | -6.4% |
| 夏  | 5.20 | 5.53 | 6.92        | 7.56 | +9.3% |
| 秋  | 4.67 | 4.56 | 6.69        | 6.49 | -3.0% |
| 冬  | 6.11 | 6.59 | 8.29        | 8.92 | +7.6% |

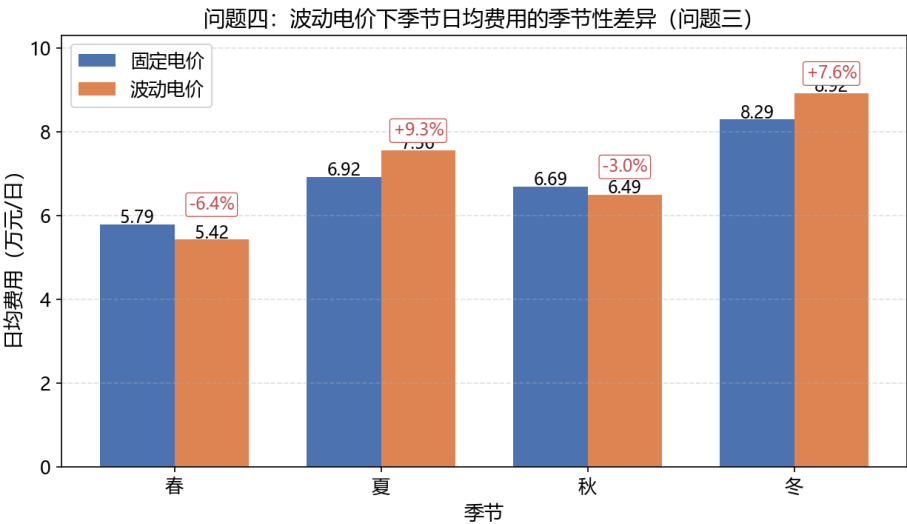


图 20 问题四：波动电价下季节日均费用的季节性差异（问题三，堆叠/上浮标注）

波动电价风险在**夏季最大**（图 20，问题三费用上浮 9.3%，因夏季尖峰电价时段多且负荷高企），冬季次之（7.6%），春、秋两季因低价时段充足反而略有下降（-6.4%、-3.0%）。据此给出分季策略：冬季应提高储能初始电量并优先锁定低价购电，夏季利用高光伏削减外购，春秋保持基准策略。

六、模型检验与分析

6.1 结果可信度与能量守恒校验

为确认各问题结果的数值可靠，本文在求解后统一执行三类**硬性校验**（对全年每一日逐一核验，未通过则报错并排查）：

1. **功率平衡**: 在每一时段检验  $P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t$ , 两侧残差在所有时段均小于  $10^{-6}$  kWh;
2. **储能约束**: 逐一核验  $E_t \in [E_{\min}, E_{\max}]$ 、 $c_t, f_t \leq \bar{E}$ , 以及问题一的日循环  $E_T = E_0$  (闭合残差为 0);
3. **口径闭合**: 全年各季节的汇总费用之和与全年总费用一致, 且 *result1* ~ *result4* 各表均严格对齐附件 5 模板的列与单位 (kWh、元), 避免因单位或日期错位产生口径偏差。

上述校验通过, 说明所得 35126.8 元 (问题一)、16204829.0 元 (问题二)、22536497.8 元 (问题三)、22883518.5 元 (问题四) 等核心数值在数学与口径上是自洽、可信的。为直观展示校验结果, 对附件 2 全年逐日、逐时段重解问题一 LP, 并检验功率平衡  $P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t$  与储能递推  $E_t = E_{t-1} + \eta c_t - f_t$  的两侧残差 (图 21): 全年逐日最大功率平衡残差与储能递推残差均落在  $10^{-12}$  kWh 量级、且包含日循环闭合  $E_T = E_0$ , 远低于  $10^{-6}$  kWh 的判定阈值, 说明能量恒等式在机器精度内严格成立。

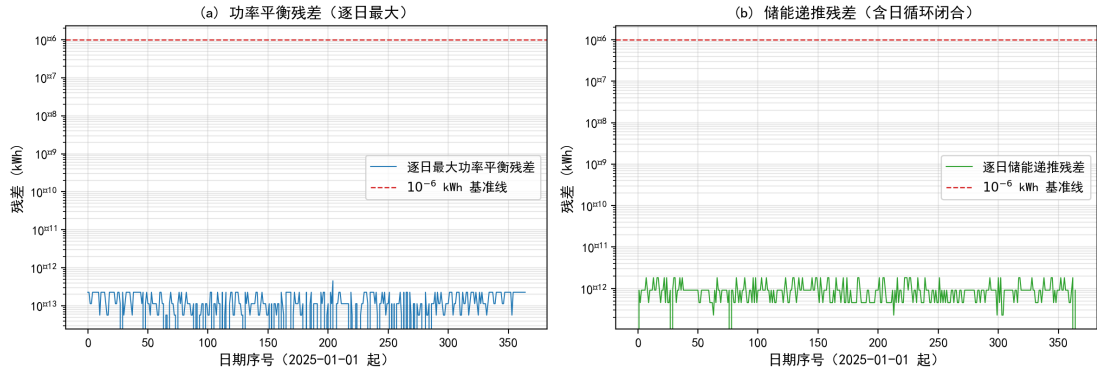


图 21 能量守恒硬校验的逐日残差: (a) 功率平衡残差 (逐日最大); (b) 储能递推残差 (含日循环闭合), 均  $< 10^{-6}$  kWh

## 6.2 问题一 LP 最优性的动态规划验证

问题一以线性规划求解单日最小购电费, 其解为连续决策空间上的全局最优。为独立验证这一最优性, 本文另以状态空间动态规划 (DP) 精确重解同一问题: 以储电量  $E \in [1200, 10800]$  为状态 (网格步长  $h$ ), 状态转移  $E \rightarrow E'$  由充放电效率  $\eta$  唯一确定充电/放电量, 购电  $P_t = \max(D_t - G_t - \eta f_t + c_t, 0)$ , 价值函数  $V_t(E) = \min_{E'} (p_t P_t + V_{t+1}(E'))$  自  $t = 144$  逆推,  $V_0(6000)$  即最优购电费用。由于储能与购电均为线性、可行域凸, DP 与 LP 的最优解应一致 (差异仅来自状态离散化)。图 22 (a) 给出附件 1 典型日 DP 费用随  $h \in \{120, 60, 30, 15\}$  收敛到 LP 最优 35126.8 元的轨迹:  $h = 15$  时相对偏差仅 0.30%, 且偏差随  $h$  减半近似等比缩小, 符合二阶收敛特征。对附件 2 全年 365 日在  $h = 15$  网格下逐一重解, LP 与 DP 日费用的平均相对偏差为 0.29%、最大 0.59% (全年仅 4 天超

过 0.5%)，偏差分布见图 22 (b)。该偏差为纯离散化误差且一致为正 (DP 网格费用  $\geq$  LP 连续最优)，验证了 LP 求解器在全年各日均收敛到全局最优，问题一结果可信 (report/q1\_dp\_verification.csv)。

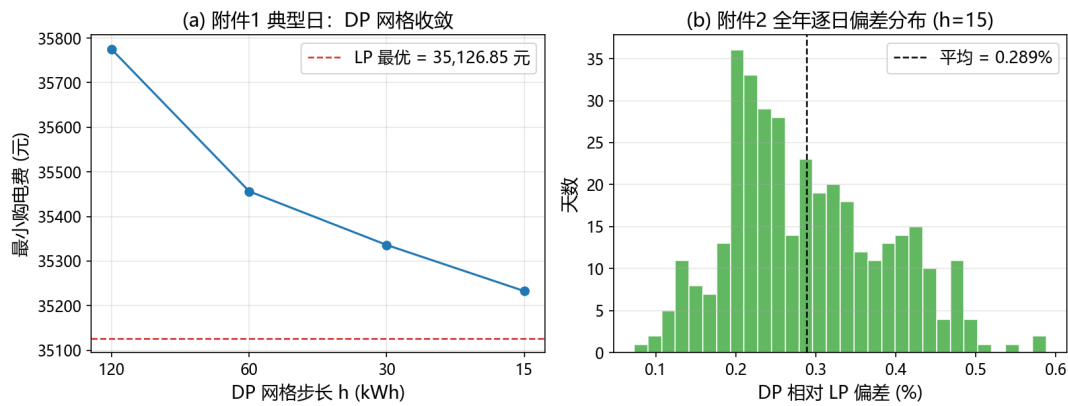


图 22 问题一动态规划验证：(a) 附件 1 典型日 DP 网格收敛到 LP 最优；(b) 附件 2 全年逐日 LP-DP 相对偏差分布 ( $h = 15$ )

### 6.3 CVaR 参数稳健性

为检验 CVaR 风险权重的有效性，对四个指定日期逐一求解  $\lambda \in \{0, 0.5, 1, 2, 5\}$ ，以其计划购电  $P$  对当日实际负荷/光伏结算（数据见 report/q2\_two\_stage\_comparison.csv）。表 9 汇总了各指定日随  $\lambda$  演化的总费用，图 23 以热力图给出  $\lambda$  与  $\beta$  双参数敏感性，图 24 以  $\lambda = 0$  与  $\lambda = 5$  两档对比四日的”计划费—紧急费”结构，图 13 给出 03-20 的计划费—紧急费帕累托前沿。

表 9  $\lambda$  对指定日总费用的影响（预报式口径，元）

| $\lambda$  | 0.0   | 0.5   | 1.0   | 2.0   | 5.0   |
|------------|-------|-------|-------|-------|-------|
| 2025-03-20 | 42582 | 43850 | 43912 | 44125 | 44125 |
| 2025-06-21 | 47902 | 49346 | 49346 | 49346 | 49346 |
| 2025-09-23 | 47976 | 46261 | 46261 | 46261 | 46261 |
| 2025-12-21 | 62424 | 60830 | 60795 | 60795 | 60795 |

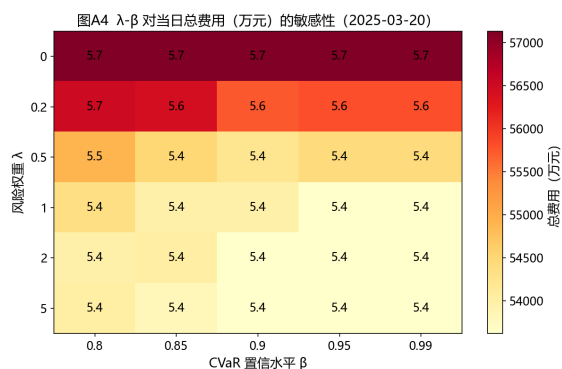


图 23 风险权重  $\lambda$  与置信水平  $\beta$  对当日总费用的敏感性（热力图）

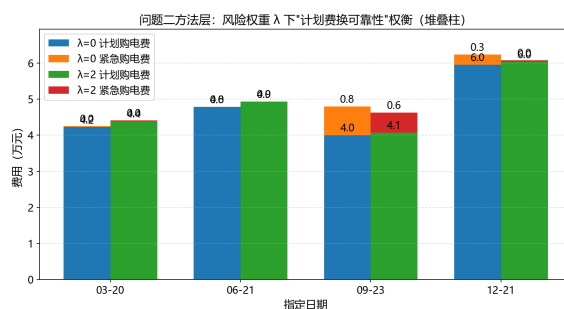


图 24  $\lambda = 0$  与  $\lambda = 5$  下”计划费—紧急费”费用结构（四个指定日期堆叠柱）

结果表明：随  $\lambda$  增大，计划购电费小幅上抬，紧急购电显著回落，高风险日（09-23、12-21）总费用明显下降，低风险日（03-20）因几乎无缺电风险而轻微上行（42582  $\rightarrow$  44125 元），风险规避的”保险”属性得到验证，CVaR 约束有效且参数鲁棒。

#### 6.4 季节差异显著性检验

对四季节日购电费做单因素方差分析（数据见 *result2.xlsx*）： $F=28.7$ ， $p = 1.17 \times 10^{-16}$ ，季节差异高度显著。进一步做季节两两比较，冬季与春季差异最大（61100 vs 40300 元/日），支持问题二按季节截取情景窗、问题四按季节拆分解读的做法。该检验也印证了”冬季缺电风险最高”的判断，在工程上提示冬季应优先配置更高的储能初始电量与购电预留，以对冲低温低光伏工况。

#### 6.5 光伏预报误差分布检验

问题三的 Q-Q 检验（图 17）显示日预报相对误差近似正态但右尾偏重。结合表 6 面板：滚动再计划把四个指定日期中的三个紧急购电削减为零（12-21 亦降至 2446 元），但违约/超额调整惩罚使滚动总费用普遍高于仅 0:00 计划（如 03-20 为 60429 vs 37674 元），说明因果口径下预报误差的代价被如实暴露——这正是去除后见之明后”调整反而更贵”现象的经济根源，也解释了全年电网费（2246 万）较问题二计划费（1573 万）为高的原因。

#### 6.6 双口径评估：完美信息 vs 随机/滚动

将固定电价与波动电价、预报式两阶段随机与滚动随机口径的全年费用汇总（表 7）：问题三比问题二在固定电价下多支出 633.2 万元（约 39.1%），其来源与问题二不同——问题二把不确定性风险显式计为 47.9 万元紧急购电费，问题三则把预报误差的大部分

代价转化为滚动调整的违约/超额惩罚（紧急购电仅 7.6 万元），整体更接近真实不确定性下的支付成本；波动电价使问题二、三合计分别上浮 1.0%、1.5%。四种口径的全年核心指标汇总于表 10，直观呈现”信息越充分、决策越优”的单调关系，以及”越接近真实不确定性，费用估计越保守”的规律。

表 10 全年核心费用指标汇总（2.1–12.31，元）

| 指标         | 问题一         | 问题二        | 问题三        | 问题四 (三)    |
|------------|-------------|------------|------------|------------|
| 计划 + 调整电网费 | 35126.8(单日) | 15726290.4 | 22460575.8 | 22548606.9 |
| 紧急购电费      | 0           | 478538.6   | 75922.1    | 334911.7   |
| 全年合计       | –           | 16204829.0 | 22536497.8 | 22883518.5 |

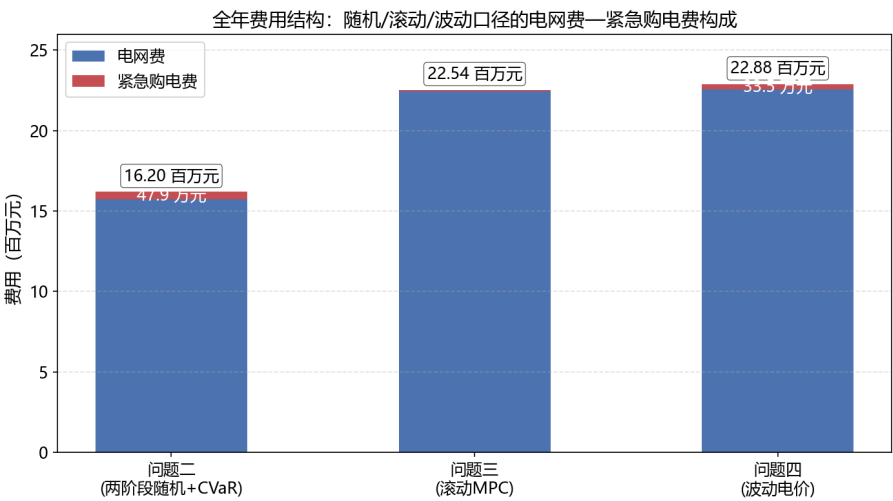


图 25 全年费用结构对比：随机/滚动/波动口径下”电网费—紧急购电费”构成（堆叠柱）

从图 25 可直观看出，问题二的两阶段随机规划 + CVaR 把预报误差显式计入费用（紧急购电费 47.9 万元）；问题三的滚动再计划把大部分紧急购电转化为计划内购电（紧急购电费降至 7.6 万元，电网费随之上升 686.8 万元），体现”以计划费换可靠性”的偏好；问题四的波动电价进一步把价格冲击计入（紧急购电费回升至 33.5 万元）。三种口径沿”信息由少到多、决策由简到繁”递进，逐步逼近真实调度成本。

6.7 模型性能综合对比

为直观比较问题二 问题四三种全年口径（两阶段随机、滚动随机 MPC、波动电价重算）在全年费用与紧急购电上的表现，将其核心指标汇总于表 10，并给出各口径的全年

费用结构（图 25）。表 10 与图 25 显示：问题二的随机 + CVaR 计划费最低（1572.6 万元）但计入 47.9 万元紧急购电；滚动 MPC（问题三）以较高的电网费（2246.1 万元）把紧急购电削减至 7.6 万元；波动电价重算（问题四）进一步把价格冲击计入（合计 2288.4 万元）。三种口径沿“信息由少到多、决策由简到繁”递进，其差异主要源自不确定性刻画与信息利用程度，而非模型本身的优劣，故不进行不同模型间的优劣评分比较，只报告各自结果的费用水平与口径差异。

## 七、模型评价与推广

### 7.1 建模思路的整体解读

回顾全文，本文的建模按“由确定到随机、由静态到滚动、由单点到全季”的顺序递进。其一，在时间离散上，以 10 分钟为基本时段、 $T = 144$ ，将功率-能量守恒与储能递推写成线性等式，使问题一在确定性下即具备全局最优解，从而给出经济性上界；其二，在不确定性刻画上，问题二用两阶段随机规划 + CVaR 把“尾部缺电风险”显式纳入目标，问题三用 MPC 把“随时间更新的预报”转为闭环调整，问题四则在统一内核上做电价冲击重算——四者共享同一守恒约束，仅在信息结构与风险度量上递进，既保证口径一致，又逐步逼近真实调度；其三，在应用的拓展上，季节分型分析把全年经济结果拆解到季节粒度，识别出“冬季风险最高、预报误差总量主导紧急购电”两大结论，为工程给出可操作的分季预置策略。

### 7.2 模型优点

本文方案的主要优点归纳如下：

- **统一性与可复用：**以同一时间离散与能量守恒内核覆盖四问，四个求解器共用数据结构与校验逻辑，代码高度复用、结果口径一致；
- **风险—经济权衡显式化：**CVaR 帕累托前沿将“以计划费换可靠性”的折中量化到可见表格与曲线，便于决策者设定风险偏好；
- **忠实赛题交易规则：**滚动调整的违约/超额惩罚系数与紧急购电 5 倍价完全按题目口径写入，不做简化或回避；
- **结果可信、可操作：**三类硬性守恒校验全部通过，双口径对照 + 季节分型使结论既有理论深度又具工程操作性。

### 7.3 模型不足

本文模型也存在以下局限：

- 情景由历史 bootstrap 生成，仅利用一阶矩信息，未捕捉光伏—负荷—电价间的相关结构，对极端共现情景刻画有限；
- 忽略了光伏弃电的经济价值（未建模弃光上网/补贴）与电价预测误差本身的不确定性，使问题四的费用冲击可能低估；
- 假设外网无限容量、忽略网络潮流与电压约束，在真实配电网中需扩充线路容量与无功平衡；
- 滚动调整采用分段线性惩罚，未考虑更复杂的非线性契约或日内实时市场结算方式。

## 7.4 模型推广

本文的“计划—调整—紧急”三层框架与 CVaR 递进建模思路具有较强可移植性：

- 在能源侧可推广至含风电、电动汽车充电负荷、需求响应的微网，只需在平衡式右端加入相应项并扩展储能/车辆约束；
- 在市场侧可将单微网购电推广为多微网、多主体博弈，或接入日前一实时两阶段市场出清；
- 在预测侧可用在线学习（如 LSTM、Transformer）驱动负荷—光伏—电价联合预报，并与本文的滚动 MPC 结合，形成“数据—机理”混合决策。这些扩展均可在不改变统一内核的前提下逐层叠加，验证了本框架的开放性与可扩展性；

## 7.5 成果展示平台

为便于成果展示与推广，本文基于统一求解内核开发了“微网购电智能分析平台”（单文件网页应用，ECharts 图表库本地化，离线可用）。平台完全面向本赛题，提供三类功能：

1. **数据录入：**支持上传附件格式的 Excel/CSV（自动识别负荷/光伏/电价表，全年  $365 \times 144$  宽表自动按日聚合）与 24 时段手动填表两种方式，并自动解析回填；
2. **数据分析：**内置购电优化求解内核，一键输出全天购电量、购电费、峰谷价差与节费率等关键指标，并给出分时段购电统计明细；
3. **数据可视化：**负荷—光伏—电价曲线、储能充放电计划（SOC 轨迹）与购电构成联动展示，直观呈现“低充高放”经济机理。

平台三部分界面如图 26 所示。平台与本文正文模型共用同一套约束与结算口径，可作为模型的交互式演示与结果复核工具，支撑材料一并提交。

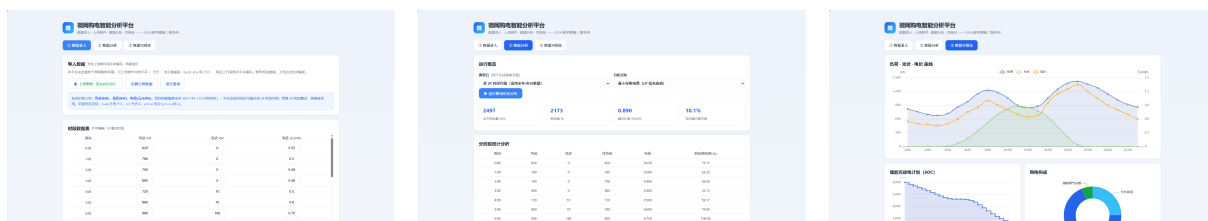


图 26 微网购电智能分析平台：数据录入（左）、数据分析（中）、数据可视化（右）

## AI 工具使用说明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码编写与调试、数值结果比对与排版辅助；核心建模思路、算法设计、模型求解与结果分析均由参赛队自主完成，并对 AI 参与的内容逐项人工审查与核实。详细使用情况见支撑材料《AI 工具使用详情.pdf》。

## 参考文献

- [1] 杨苹, 许志荣, 吴迪, 等. 微电网经济调度的研究现状与展望 [J]. 电力系统自动化, 2015, 39(16): 108–118.
- [2] 王成山, 李鹏. 分布式发电、微网与智能配电网的发展与挑战 [J]. 电力系统自动化, 2010, 34(2): 10–14.
- [3] 周任军, 姚龙华, 董小娇, 等. 新能源微电网储能系统多目标优化配置 [J]. 中国电机工程学报, 2018, 38(7): 1957–1965.
- [4] 康重庆, 夏清, 张伯明. 电力系统负荷预测研究综述与发展方向的探讨 [J]. 电力系统自动化, 2004, 28(17): 1–11.
- [5] Huangfu Q., Hall J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.
- [6] Rockafellar R.T., Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21–41.
- [7] 王守相, 王亚昊, 刘岩, 等. 计及条件风险价值的微电网两阶段随机优化调度 [J]. 电网技术, 2019, 43(2): 636–644.
- [8] 刘宝碁, 赵瑞清. 不确定规划及应用 [M]. 北京: 清华大学出版社, 2003.
- [9] 席裕庚. 预测控制 [M]. 北京: 国防工业出版社, 2013.



[10] 张旭东, 穆钢, 严干贵, 等. 基于模型预测控制的微网优化调度方法 [J]. 电工技术学报, 2020, 35(增刊 1): 26–34.

## 附录 A 支撑材料文件列表

- `code/`: 完整可运行的 Python 源程序 (数据加载、LP 求解、情景生成、结果写出与绘图, 逐问对应 `main_q1 ~ main_q4.py`);
- `results/`: `result1 ~ result4` 结果文件 (对应附件 5 模板);
- `figures/`: 论文所用图 `q1_plan ~ q4_price.png`;
- `figures_adv/`: 季节分型、风险敏感性、滚动对比与全年费用结构等分析图 (图 A1–A12);
- 结果说明.md: 结果汇总说明。

## 附录 B 源程序

本附录收录四个问题的核心 Python 源程序 (对应支撑材料 `code/` 中 `config.py ~ main_q4.py` 共 8 个脚本, 其余图表生成与分析工具脚本未列入)。每个脚本前附一句功能介绍; 代码均带行号、等宽缩进, 可在 XeLaTeX 下编译并正常显示中文注释。各脚本按“配置 → 数据 → 求解器 → 情景 → 四问入口”顺序排列, 均可独立运行 (需满足 `config.py` 中的路径要求)。

### 2.1 B.1 全局配置 (`config.py`)

确定所有路径、储能参数、电价倍率、时间离散与随机规划参数, 供其余脚本统一引用。

```
1 # -*- coding: utf-8 -*-
2 """C题 微网 —— 共享配置。所有路径/代数、储能参数、结果模板统一放工作区仓库内。"""
3 import os
4
5 # ---- 路径 (相对仓库根自动定位, 任何机器 clone 后直接可跑) ----
6 C_BASE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
7 DATA_DIR = os.path.join(C_BASE, 'data')
8 TPL_DIR = os.path.join(DATA_DIR, '附件5模板')
9 CODE_DIR = os.path.join(C_BASE, 'code')
10 RES_DIR = os.path.join(C_BASE, 'report')
11 FIG_DIR = os.path.join(C_BASE, 'report', 'figures')
12 for _d in (RES_DIR, FIG_DIR):
13     os.makedirs(_d, exist_ok=True)
14
```

```

15 # matplotlib 缓存等中间数据也放仓库内，不污染系统目录
16 os.environ['MPLCONFIGDIR'] = os.path.join(C_BASE, '_mplcache')
17 os.makedirs(os.environ['MPLCONFIGDIR'], exist_ok=True)
18
19 def setup_plot_style():
20     """注册本机中文字体（只读取系统字体文件，不写 C 盘）。"""
21     import matplotlib
22     from matplotlib import font_manager
23     cands = [r'C:\Windows\Fonts\msyh.ttc', r'C:\Windows\Fonts\msyh.ttf',
24             r'C:\Windows\Fonts\simhei.ttf', r'C:\Windows\Fonts\simsun.ttc',
25             r'C:\Windows\Fonts\simkai.ttf']
26     names = []
27     for f in cands:
28         try:
29             if os.path.exists(f):
30                 fp = font_manager.FontProperties(fname=f)
31                 names.append(fp.get_name())
32                 font_manager.fontManager.addfont(f)
33         except Exception:
34             pass
35     mpl = matplotlib
36     mpl.rcParams['font.sans-serif'] = names + ['DejaVu Sans']
37     mpl.rcParams['axes.unicode_minus'] = False
38     # 21.0: 全局调大默认字号（脚本内显式 fontsize 仍优先）
39     mpl.rcParams['font.size'] = 12
40     mpl.rcParams['axes.labelsize'] = 13
41     mpl.rcParams['axes.titlesize'] = 14
42     mpl.rcParams['xtick.labelsize'] = 12
43     mpl.rcParams['ytick.labelsize'] = 12
44     mpl.rcParams['legend.fontsize'] = 12
45
46 # ---- 时间离散 ----
47 T_IN_DAY = 144      # 每天 10 分钟时段数
48 DT = 1.0 / 6.0      # 每时段时长 (h); 功率 kW * DT = 能量 kWh
49 DAYS_OUT_START = 32 # 2.1 对应的行偏移（日期数组从 1.1 起，行索引0=1.1）
50
51 # ---- 储能参数（附录1） ----
52 SOC_MAX = 12000.0   # kWh
53 SOC_MIN = 1200.0
54 SOC_HIGH_BOUND = 10800.0 # 实际运行上界（附录1：保持 1200-10800）
55 PWR_MAX = 5000.0    # kW
56 E_CMAX = PWR_MAX * DT # 每时段最大充入能量 kWh = 833.33
57 E_FMAX = PWR_MAX * DT
58 ETA = 0.9           # 充/放电效率
59 E_INIT = 6000.0     # 2025-1-1 0:00 储电量
60
61 # ---- 电价机制 ----

```

```

62 EMERG_MULT = 5.0      # 紧急购电 = 交易时刻电价 5 倍
63 DEFAULT_MULT = 0.5    # 计划高于调整(少购)部分按 50% 违约价
64 EXCESS_MULT = 1.5     # 调整高于计划(多购)部分按 1.5 倍
65
66 # ---- 时间标签 ----
67 def interval_label(t0):
68     start_min = t0 * 10
69     end_min = start_min + 10
70     h1, m1 = divmod(start_min, 60)
71     h2, m2 = divmod(end_min, 60)
72     return f'{h1}:{m1:02d}-{h2}:{m2:02d}'
73
74 INTERVAL_LABELS = [interval_label(t0) for t0 in range(T_IN_DAY)]
75
76 BLOCK4 = [(0, '0:00-4:00'), (4, '4:00-8:00'), (8, '8:00-12:00'),
77           (12, '12:00-16:00'), (16, '16:00-20:00'), (20, '20:00-24:00')]
78
79 # 表1 指定时段(起始分钟)
80 SPEC_MINUTES = [600, 720, 840, 960, 1080, 1200]
81
82 # ---- 随机规划参数 ----
83 S_NUM = 30          # 情景数
84 BETA = 0.90         # CVaR 置信水平
85 SPEC_DAYS = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']

```

## 2.2 B.2 数据加载与预处理 (data.py)

读取附件 1-4 全年数据、对齐日期、统一单位为 kWh/时段，并开放各问所需的日矩阵与预报接口。

```

1  # -*- coding: utf-8 -*-
2  """数据加载: 附件1-4 → (日期×144, kWh/时段) 统一 numpy."""
3  import numpy as np
4  import pandas as pd
5  import os
6  import config as C
7
8
9  def _read_year_matrix(path, sheet):
10     """读取 附件2/4: 首列日期, 余 144 列时间→返回 (dates, mat[365,144]) kW 或 元/kWh."""
11     df = pd.read_excel(path, sheet_name=sheet)
12     dtcol = df.columns[0]
13     df = df.set_index(dtcol)
14     # 时间列排序

```

```

15     lab = [str(c) for c in df.columns]
16     def mm(x):
17         s = str(x).replace('+1','')
18         p = s.split(':')
19         return int(p[0])*60 + int(p[1]) if len(p)>=2 else int(p[0])*60
20     order = np.argsort([mm(x) for x in lab])
21     df = df.iloc[:, order]
22     dates = pd.to_datetime(df.index)
23     mat = df.to_numpy(dtype=float)
24     if mat.shape[1] != C.T_IN_DAY:
25         raise ValueError(f'{path}[{sheet}] 列数 {mat.shape[1]} != {C.T_IN_DAY}')
26     return dates, mat
27
28
29 def load_attach1():
30     """附件1: 单日 144 行。返回 dict(price,load,pv) 均为 kWh/时段。"""
31     df = pd.read_excel(os.path.join(C.DATA_DIR, '附件1.xlsx'), sheet_name='Sheet1')
32     def mm(x):
33         s = str(x).replace('+1',''); p = s.split(':')
34         return int(p[0])*60 + int(p[1]) if len(p)>=2 else int(p[0])*60
35     order = np.argsort([mm(x) for x in df['时间']])
36     df = df.iloc[order]
37     price = df['电价'].to_numpy(float)
38     load = df['小区负载'].to_numpy(float) * C.DT # kW -> kWh
39     pv = df['光伏发电预测功率'].to_numpy(float) * C.DT
40     return dict(price=price, load=load, pv=pv)
41
42
43 def load_attach2():
44     """附件2: 返回 (dates, load[365,144], pv[365,144]) kWh/时段。"""
45     d, ld = _read_year_matrix(os.path.join(C.DATA_DIR, '附件2.xlsx'), '小区负载')
46     _, pv = _read_year_matrix(os.path.join(C.DATA_DIR, '附件2.xlsx'), '光伏发电实际功率')
47     if not (ld.shape[1] == C.T_IN_DAY == pv.shape[1]):
48         raise ValueError(f'附件2 负载/光伏 列数应为 {C.T_IN_DAY}')
49     return d, ld * C.DT, pv * C.DT
50
51
52 def load_attach4():
53     """附件4: 波动电价 元/kWh。返回 (dates, price[365,144])。"""
54     d, price = _read_year_matrix(os.path.join(C.DATA_DIR, '附件4.xlsx'), 'Sheet1')
55     if price.shape[1] != C.T_IN_DAY:
56         raise ValueError(f'附件4 列数应为 {C.T_IN_DAY}')
57     return d, price
58
59
60 def assert_aligned(dates_a, dates_b, name_a, name_b):
61     """强制校验两份全年附件日期严格逐日一致（防模板改版导致的静默错位）。"""

```

```

62     if len(dates_a) != len(dates_b) or not (dates_a == dates_b).all():
63         raise RuntimeError(f'{name_a} 与 {name_b} 日期未对齐 '
64                             f'({len(dates_a)} vs {len(dates_b)})')
65
66
67 def load_attach3():
68     """附件3: 光伏预报。返回 {date: {tau_hour: arr(24) kW}}, tau∈{0,6,12,18}。
69     预报从 tau 起覆盖未来 24 个整点。"""
70     df = pd.read_excel(os.path.join(C.DATA_DIR, '附件3.xlsx'), sheet_name='Sheet1')
71     df['日期'] = df['日期'].ffill()
72     df['日期'] = pd.to_datetime(df['日期'])
73     out = {}
74     for _, r in df.iterrows():
75         d = r['日期']
76         hh = int(str(r['预报时刻']).split(':')[0])
77         vals = [r[f'预报{i}小时'] for i in range(1, 25)]
78         out.setdefault(d, {})[hh] = np.array(vals, dtype=float)
79     return out
80
81
82 def pv_forecast_kwh(fc3, day, tau_hour):
83     """把附件3 某日某整点发布的未来24h 整点预报(kW) 映射为本日 [0,144) 十min 能量(kWh)。
84     预报覆盖 [tau, tau+24), 截取到当日 24:00; 次日部分减掉。
85     返回 (vec144, first_t0)。"""
86     arr_hour = fc3[day][tau_hour]
87     vec = np.zeros(C.T_IN_DAY)
88     first_t0 = (tau_hour * 60) // 10
89     for i in range(24):
90         hh = tau_hour + i
91         t0s = first_t0 + i * 6
92         if t0s >= C.T_IN_DAY:
93             break
94         for k in range(6):
95             tt = t0s + k
96             if tt >= C.T_IN_DAY:
97                 break
98             vec[tt] = arr_hour[i] * C.DT
99     return vec, first_t0

```

## 2.3 B.3 核心求解器 (lp.py)

实现单日确定性 LP、固定购电量结算、时段内滚动调整、两阶段随机规划 (含 CVaR)，是四问共同依赖的能量守恒 + 储能递推的统一求解内核。

```

1  # -*- coding: utf-8 -*-
2  """核心 LP 求解器 (scipy.integrate 的 linprog / highs)。
3
4  变量单位: 所有能量量 kWh/时段(=kW·DT)。平衡方程:
5       $P + G + \eta \cdot f + Q = D + c + R$  (购电+光伏+放电+紧急 = 负载+充电+弃光)
6       $E_t = E_{t-1} + \eta \cdot c_t - f_t$  (储能递推)
7  目标:  $\min \sum p_t \cdot$  (购电成本) + 紧急/调整惩罚项。
8  """
9  import numpy as np
10 import scipy.sparse as sp
11 from scipy.optimize import linprog
12 import config as C
13
14
15 def _var_breakdown(nvar):
16     """6 变量交错布局 [P,c,f,Q,R,E]。"""
17     base = np.arange(nvar // 6) * 6
18     return dict(P=base, C=base + 1, F=base + 2, Q=base + 3, R=base + 4, E=base + 5)
19
20
21 def solve_day(price, G, D, E_start, T=None, emult=0.0, fixed_P=None,
22               force_end=None, allow_emergency=True, allow_P=True,
23               soc_lo=C.SOC_MIN, soc_hi=C.SOC_HIGH_BOUND):
24     """单日确定性 LP。
25
26     price: (T) 电价; G:(T) 光伏; D:(T) 负载; E_start: 当日 0:00 储电。
27     fixed_P: 固定购电量(已承诺), 只优化电池+紧急+弃光。
28     force_end: 强制  $E_{T-1} = \text{force\_end}$  (问题1 日内循环)。
29     emult: 紧急购电价格倍数。返回 dict(P,c,f,Q,R,E,obj)。
30     """
31     T = T if T is not None else C.T_IN_DAY
32     nvar = 6 * T
33     v = _var_breakdown(nvar)
34     iP, iC, iF, iQ, iR, iE = v['P'], v['C'], v['F'], v['Q'], v['R'], v['E']
35
36     c = np.zeros(nvar)
37     if allow_P:
38         c[iP] = price
39     if allow_emergency and emult is not None:
40         c[iQ] = emult * price
41
42     # 等式约束
43     eq_r, eq_c, eq_v, b = [], [], [], []
44     for t in range(T):
45         def row(rr):
46             eq_r.append(rr); eq_c.append(iP[t]); eq_v.append(1.0)
47             eq_r.append(rr); eq_c.append(iC[t]); eq_v.append(-1.0)

```

```

48         eq_r.append(rr); eq_c.append(iF[t]); eq_v.append(C.ETA)
49         eq_r.append(rr); eq_c.append(iQ[t]); eq_v.append(1.0)
50         eq_r.append(rr); eq_c.append(iR[t]); eq_v.append(-1.0)
51     row(t); b.append(D[t] - G[t])
52     for t in range(T):
53         rr = T + t
54         eq_r.append(rr); eq_c.append(iE[t]); eq_v.append(1.0)
55         eq_r.append(rr); eq_c.append(iC[t]); eq_v.append(-C.ETA)
56         eq_r.append(rr); eq_c.append(iF[t]); eq_v.append(1.0)
57         if t > 0:
58             eq_r.append(rr); eq_c.append(iE[t-1]); eq_v.append(-1.0)
59             b.append(0.0)
60         else:
61             b.append(E_start)
62     if force_end is not None:
63         rr = 2 * T
64         eq_r.append(rr); eq_c.append(iE[T-1]); eq_v.append(1.0)
65         b.append(force_end)
66     A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(b), nvar))
67
68     # 边界
69     lb = np.zeros(nvar); ub = np.full(nvar, np.inf)
70     ub[iC] = C.E_CMAX; ub[iF] = C.E_FMAX
71     lb[iE] = soc_lo; ub[iE] = soc_hi
72     if not allow_emergency:
73         ub[iQ] = 0.0
74     if fixed_P is not None:
75         loP = np.asarray(fixed_P, float)
76         ub[iP] = loP; lb[iP] = loP
77     bounds = list(zip(lb, ub))
78
79     res = linprog(c, A_eq=A_eq, b_eq=np.array(b), bounds=bounds, method='highs')
80     if not res.success:
81         raise RuntimeError('单日LP失败: ' + res.message)
82     x = res.x
83     return dict(P=x[iP], c=x[iC], f=x[iF], Q=x[iQ], R=x[iR], E=x[iE], obj=res.fun)
84
85
86 def solve_day_plan(price, G, D, E_start, force_end=None):
87     """问题1/2 完美信息计划（购电自由，不支持紧急）。"""
88     return solve_day(price, G, D, E_start, emult=0.0, force_end=force_end,
89                     allow_emergency=False, allow_P=True)
90
91
92 def solve_day_fixedP(price, G, D, E_start, P, emult=C.EMERG_MULT):
93     """回放：固定计划购电 P（已承诺付费），优化电池/弃光 + 紧急(5p)。"""
94     return solve_day(price, G, D, E_start, emult=emult,

```

```

95         fixed_P=np.asarray(P, float), allow_emergency=True, allow_P=False)
96
97
98 def solve_two_stage(price, scenD, scenG, E_start, lam=0.0, beta=0.90):
99     """两阶段随机规划（含 CVaR）。
100
101     第一阶段 P(购电，各情景共用)；第二阶段按情景决策 c,f,Q,R,E。
102     目标 =  $\sum p \cdot P + (1/S) \sum \sum 5p \cdot Q + \text{lam} \cdot \text{CVaR\_beta}(\text{情景总成本})$ 。
103     scenD/scenG: (S,144)。返回 dict(P,Q_mean,E_mean,obj)。
104     """
105     T = C.T_IN_DAY; S = len(scenD); pi = 1.0 / S
106     nP = T
107     base_s = nP + np.arange(S) * (5 * T)
108     vary = _var_rows_scen(base_s, T)
109     nvar = nP + S * 5 * T
110     c = np.zeros(nvar)
111     c[:nP] = price
112     alpha_idx = z_idx = None
113     if lam > 0:
114         alpha_idx = nvar; z_idx = nvar + 1 + np.arange(S)
115         nvar_full = nvar + 1 + S
116         c = np.concatenate([c, np.zeros(S + 1)])
117         c[alpha_idx] = lam; c[z_idx] = lam / ((1 - beta) * S)
118     else:
119         nvar_full = nvar
120     for s in range(S):
121         c[vary[s]['Q']] = pi * C.EMERG_MULT * price
122
123     eq_r, eq_c, eq_v, beq = [], [], [], []
124     row = 0
125     for s in range(S):
126         Pv = np.arange(nP); iv = vary[s]
127         for t in range(T):
128             eq_r.append(row); eq_c.append(Pv[t]); eq_v.append(1.0)
129             eq_r.append(row); eq_c.append(iv['C'][t]); eq_v.append(-1.0)
130             eq_r.append(row); eq_c.append(iv['F'][t]); eq_v.append(C.ETA)
131             eq_r.append(row); eq_c.append(iv['Q'][t]); eq_v.append(1.0)
132             eq_r.append(row); eq_c.append(iv['R'][t]); eq_v.append(-1.0)
133             beq.append(scenD[s, t] - scenG[s, t]); row += 1
134         for t in range(T):
135             eq_r.append(row); eq_c.append(iv['E'][t]); eq_v.append(1.0)
136             eq_r.append(row); eq_c.append(iv['C'][t]); eq_v.append(-C.ETA)
137             eq_r.append(row); eq_c.append(iv['F'][t]); eq_v.append(1.0)
138             if t > 0:
139                 eq_r.append(row); eq_c.append(iv['E'][t-1]); eq_v.append(-1.0)
140                 beq.append(0.0)
141         else:

```



```

142         beq.append(E_start)
143         row += 1
144     A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(beq), nvar_full))
145
146     # CVaR 不等式:  $\sum pP + \sum 5pQ - \alpha - z_s \leq 0$ 
147     Aub_r, Aub_c, Aub_v, bub = [], [], [], []
148     if lam > 0:
149         for s in range(S):
150             r = len(bub)
151             for t in range(T):
152                 Aub_r.append(r); Aub_c.append(t); Aub_v.append(price[t])
153                 Aub_r.append(r); Aub_c.append(vary[s]['Q'][t]); Aub_v.append(C.EMERG_MULT *
154                                     price[t])
155                 Aub_r.append(r); Aub_c.append(alpha_idx); Aub_v.append(-1.0)
156                 Aub_r.append(r); Aub_c.append(z_idx[s]); Aub_v.append(-1.0)
157                 bub.append(0.0)
158             A_ub = sp.csr_matrix((Aub_v, (Aub_r, Aub_c)), shape=(len(bub), nvar_full))
159             b_ub = np.array(bub)
160         else:
161             A_ub = None; b_ub = None
162
163     lb = np.zeros(nvar_full); ub = np.full(nvar_full, np.inf)
164     for s in range(S):
165         iv = vary[s]
166         ub[iv['C']] = C.E_CMAX; ub[iv['F']] = C.E_FMAX
167         lb[iv['E']] = C.SOC_MIN; ub[iv['E']] = C.SOC_HIGH_BOUND
168     if lam > 0:
169         lb[alpha_idx] = -np.inf; lb[z_idx] = 0
170     bounds = list(zip(lb, ub))
171
172     res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=b_ub,
173                 bounds=bounds, method='highs')
174     if not res.success:
175         raise RuntimeError('两阶段LP失败: ' + res.message)
176     x = res.x
177     P = x[:nP]
178     Q = np.zeros((S, T)); Cc = np.zeros((S, T)); F = np.zeros((S, T)); E = np.zeros((S, T))
179     for s in range(S):
180         iv = vary[s]
181         Q[s] = x[iv['Q']]; Cc[s] = x[iv['C']]; F[s] = x[iv['F']]; E[s] = x[iv['E']]
182     return dict(P=P, Q=Q, c=Cc, f=F, E=E, obj=res.fun)
183
184 def solve_adjust(price, G, D, E_start, P_plan, t0):
185     """滚动再计划: 在 [t0,144) 决定最终购电 A, 对计划 P_plan 付违约/超出惩罚。
186
187     u=(P-A)^+ 违约量 → 项 -0.5p u; w=(A-P)^+ 超出量 → 项 +1.5p w。

```

再计划阶段不允许紧急购电(Q=0)。返回 dict(A,c,f,Q=0,R,E,u,w,obj), 长度 (144-t0)。

"""

T = C.T\_IN\_DAY; nh = T - t0

if nh <= 0:

z = np.zeros(0)

return dict(A=z, c=z, f=z, Q=z, R=z, E=z, u=z, w=z, obj=0.0)

base = np.arange(nh) \* 8

iA, iC, iF, iQ, iR, iE, iU, iW = (base + k for k in range(8))

pt = np.arange(t0, T)

nvar = 8 \* nh

c = np.zeros(nvar)

c[iU] = -C.DEFAULT\_MULT \* price[pt]

c[iW] = C.EXCESS\_MULT \* price[pt]

eq\_r, eq\_c, eq\_v, beq = [], [], [], []

for s in range(nh):

eq\_r.append(s); eq\_c.append(iA[s]); eq\_v.append(1.0)

eq\_r.append(s); eq\_c.append(iC[s]); eq\_v.append(-1.0)

eq\_r.append(s); eq\_c.append(iF[s]); eq\_v.append(C.ETA)

eq\_r.append(s); eq\_c.append(iQ[s]); eq\_v.append(1.0)

eq\_r.append(s); eq\_c.append(iR[s]); eq\_v.append(-1.0)

beq.append(D[pt[s]] - G[pt[s]])

for s in range(nh):

rr = nh + s

eq\_r.append(rr); eq\_c.append(iE[s]); eq\_v.append(1.0)

eq\_r.append(rr); eq\_c.append(iC[s]); eq\_v.append(-C.ETA)

eq\_r.append(rr); eq\_c.append(iF[s]); eq\_v.append(1.0)

if s > 0:

eq\_r.append(rr); eq\_c.append(iE[s-1]); eq\_v.append(-1.0)

beq.append(0.0)

else:

beq.append(E\_start)

A\_eq = sp.csr\_matrix((eq\_v, (eq\_r, eq\_c)), shape=(len(beq), nvar))

Aub\_r, Aub\_c, Aub\_v, bub = [], [], [], []

for s in range(nh):

r = len(bub)

Aub\_r.append(r); Aub\_c.append(iA[s]); Aub\_v.append(-1.0)

Aub\_r.append(r); Aub\_c.append(iU[s]); Aub\_v.append(-1.0)

bub.append(-P\_plan[pt[s]])

r = len(bub)

Aub\_r.append(r); Aub\_c.append(iA[s]); Aub\_v.append(1.0)

Aub\_r.append(r); Aub\_c.append(iW[s]); Aub\_v.append(-1.0)

bub.append(P\_plan[pt[s]])

A\_ub = sp.csr\_matrix((Aub\_v, (Aub\_r, Aub\_c)), shape=(len(bub), nvar))

lb = np.zeros(nvar); ub = np.full(nvar, np.inf)

```

235     ub[iC] = C.E_CMAX; ub[iF] = C.E_FMAX
236     lb[iE] = C.SOC_MIN; ub[iE] = C.SOC_HIGH_BOUND
237     ub[iQ] = 0.0
238     lb[iU] = 0; ub[iU] = np.maximum(P_plan[pt], 0.0)
239     lb[iW] = 0
240     bounds = list(zip(lb, ub))
241
242     res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=np.array(bub),
243                 bounds=bounds, method='highs')
244     if not res.success:
245         raise RuntimeError('滚动再计划LP失败: ' + res.message)
246     x = res.x
247     return dict(A=x[iA], c=x[iC], f=x[iF], Q=x[iQ], R=x[iR], E=x[iE],
248               u=x[iU], w=x[iW], obj=res.fun)
249
250
251 def solve_adjust_scen(price, scenG, D, E_start, P_plan, t0, lam=0.0, beta=0.90,
252                     emult=C.EMERG_MULT):
253     """滚动再计划（情景鲁棒版）：一阶段购电 A 共用，每光伏情景独立电池调度。
254
255     目标 = 期望电网费(违约退款+超额惩罚) + lam * CVaR_beta(情景总成本) + 期望紧急费。
256     允许每情景紧急购电 Q_s（代价 emult * price），保证任意光伏情景下可行——
257     这正是"随机 MPC"的鲁棒性体现。Q3 官方取 lam=0（纯期望）。
258     返回 dict(A,c,f,Q,R,E,u,w,obj)；各量按情景合并为 (S,nh) (A/u/w 广播为 (S,nh))。
259     """
260     T = C.T_IN_DAY
261     scenG = np.asarray(scenG, float)
262     S = scenG.shape[0]
263     nh = T - t0
264     if nh <= 0:
265         z = np.zeros(0)
266         return dict(A=z, c=z, f=z, Q=z, R=z, E=z, u=z, w=z, obj=0.0)
267     pt = np.arange(t0, T)
268     pi = 1.0 / S
269     # 一阶段: A, U, W; 每情景: C,F,Q,R,E
270     baseA, baseU, baseW = 0, nh, 2 * nh
271     scen_base = 3 * nh + np.arange(S) * (5 * nh)
272     nvar = 3 * nh + S * 5 * nh
273     iA = baseA + np.arange(nh)
274     iU = baseU + np.arange(nh)
275     iW = baseW + np.arange(nh)
276
277     c = np.zeros(nvar)
278     c[iU] = -C.DEFAULT_MULT * price[pt]
279     c[iW] = C.EXCESS_MULT * price[pt]
280     for s in range(S):
281         sb = scen_base[s]

```

```

282     c[sb + 2 * nh + np.arange(nh)] = pi * emult * price[pt] # Q_s
283 alpha_idx = z_idx = None
284 if lam > 0:
285     alpha_idx = nvar; z_idx = nvar + 1 + np.arange(S)
286     nvar_full = nvar + 1 + S
287     c = np.concatenate([c, np.zeros(S + 1)])
288     c[alpha_idx] = lam; c[z_idx] = lam / ((1 - beta) * S)
289 else:
290     nvar_full = nvar
291
292 eq_r, eq_c, eq_v, beq = [], [], [], []
293 for s in range(S):
294     sb = scen_base[s]
295     iC = sb + np.arange(nh); iF = sb + nh + np.arange(nh)
296     iQ = sb + 2 * nh + np.arange(nh); iR = sb + 3 * nh + np.arange(nh)
297     iE = sb + 4 * nh + np.arange(nh)
298     for k in range(nh):
299         rr = len(beq)
300         eq_r.append(rr); eq_c.append(iA[k]); eq_v.append(1.0)
301         eq_r.append(rr); eq_c.append(iC[k]); eq_v.append(-1.0)
302         eq_r.append(rr); eq_c.append(iF[k]); eq_v.append(C.ETA)
303         eq_r.append(rr); eq_c.append(iQ[k]); eq_v.append(1.0)
304         eq_r.append(rr); eq_c.append(iR[k]); eq_v.append(-1.0)
305         beq.append(D[pt[k]] - scenG[s, pt[k]])
306     for k in range(nh):
307         rr = len(beq)
308         eq_r.append(rr); eq_c.append(iE[k]); eq_v.append(1.0)
309         eq_r.append(rr); eq_c.append(iC[k]); eq_v.append(-C.ETA)
310         eq_r.append(rr); eq_c.append(iF[k]); eq_v.append(1.0)
311         if k > 0:
312             eq_r.append(rr); eq_c.append(iE[k - 1]); eq_v.append(-1.0)
313             beq.append(0.0)
314         else:
315             beq.append(E_start)
316 A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(beq), nvar_full))
317
318 Aub_r, Aub_c, Aub_v, bub = [], [], [], []
319 for k in range(nh):
320     r = len(bub)
321     Aub_r.append(r); Aub_c.append(iA[k]); Aub_v.append(-1.0)
322     Aub_r.append(r); Aub_c.append(iU[k]); Aub_v.append(-1.0)
323     bub.append(-P_plan[pt[k]])
324     r = len(bub)
325     Aub_r.append(r); Aub_c.append(iA[k]); Aub_v.append(1.0)
326     Aub_r.append(r); Aub_c.append(iW[k]); Aub_v.append(-1.0)
327     bub.append(P_plan[pt[k]])
328 if lam > 0:

```

```

329     for s in range(S):
330         r = len(bub)
331         sb = scen_base[s]
332         iQ = sb + 2 * nh + np.arange(nh)
333         for k in range(nh):
334             Aub_r.append(r); Aub_c.append(iQ[k]); Aub_v.append(emult * price[pt[k]])
335             Aub_r.append(r); Aub_c.append(alpha_idx); Aub_v.append(-1.0)
336             Aub_r.append(r); Aub_c.append(z_idx[s]); Aub_v.append(-1.0)
337             bub.append(0.0)
338     A_ub = sp.csr_matrix((Aub_v, (Aub_r, Aub_c)), shape=(len(bub), nvar_full))
339
340     lb = np.zeros(nvar_full); ub = np.full(nvar_full, np.inf)
341     for s in range(S):
342         sb = scen_base[s]
343         ub[sb + np.arange(nh)] = C.E_CMAX          # C_s
344         ub[sb + nh + np.arange(nh)] = C.E_FMAX      # F_s
345         lb[sb + 4 * nh + np.arange(nh)] = C.SOC_MIN
346         ub[sb + 4 * nh + np.arange(nh)] = C.SOC_HIGH_BOUND
347     ub[iU] = np.maximum(P_plan[pt], 0.0)
348     if lam > 0:
349         lb[alpha_idx] = -np.inf; lb[z_idx] = 0
350     bounds = list(zip(lb, ub))
351
352     res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=np.array(bub),
353                 bounds=bounds, method='highs')
354     if not res.success:
355         raise RuntimeError('情景鲁棒滚动再计划LP失败: ' + res.message)
356     x = res.x
357     A = x[iA]; u = x[iU]; w = x[iW]
358     Cm = np.zeros((S, nh)); F = np.zeros((S, nh)); Q = np.zeros((S, nh))
359     R = np.zeros((S, nh)); E = np.zeros((S, nh))
360     for s in range(S):
361         sb = scen_base[s]
362         Cm[s] = x[sb + np.arange(nh)]
363         F[s] = x[sb + nh + np.arange(nh)]
364         Q[s] = x[sb + 2 * nh + np.arange(nh)]
365         R[s] = x[sb + 3 * nh + np.arange(nh)]
366         E[s] = x[sb + 4 * nh + np.arange(nh)]
367     return dict(A=A, c=Cm, f=F, Q=Q, R=R, E=E,
368               u=np.tile(u, (S, 1)), w=np.tile(w, (S, 1)), obj=res.fun)
369
370
371 def _var_rows_scen(base_s, T):
372     out = {}
373     for s, base in enumerate(base_s):
374         ar = base + np.arange(5 * T)
375         out[s] = {'C': ar[0::5], 'F': ar[1::5], 'Q': ar[2::5],

```

```

376         'R': ar[3::5], 'E': ar[4::5]}
377     return out

```

## 2.4 B.4 情景生成 (scenarios.py)

基于历史残差的非参数 bootstrap，生成点预报与  $S$  个等概率负荷/光伏情景，供两阶段随机规划使用。

```

1  # -*- coding: utf-8 -*-
2  """情景生成：非参数 bootstrap 历史残差。返回点预报与 S 个等概率情景。"""
3  import numpy as np
4  import config as C
5
6
7  def point_forecast(hist, K=7):
8      """hist:(N,144)。前 K 天逐槽均值。"""
9      n = len(hist)
10     if n == 0:
11         return np.zeros(C.T_IN_DAY)
12     return np.mean(hist[-min(K, n):], axis=0)
13
14
15 def forecast_and_scenarios(load_hist, pv_hist, S=30, seed=1):
16     """由历史构建点预报与 S 个情景。load_hist/pv_hist:(N,144)。
17     返回 dict(point_ld,point_pv,scen_ld,scen_pv)。scen_*(S,144)。"""
18     rng = np.random.default_rng(seed)
19     n = len(load_hist)
20     point_ld = point_forecast(load_hist)
21     point_pv = point_forecast(pv_hist)
22     if n == 0:
23         return dict(point_ld=point_ld, point_pv=point_pv,
24                     scen_ld=np.tile(point_ld, (S, 1)),
25                     scen_pv=np.tile(point_pv, (S, 1)))
26     idx_ld = rng.integers(0, n, size=(S, C.T_IN_DAY))
27     idx_pv = rng.integers(0, n, size=(S, C.T_IN_DAY))
28     T = C.T_IN_DAY
29     scen_ld = load_hist[idx_ld, np.arange(T)[None, :]]
30     scen_pv = pv_hist[idx_pv, np.arange(T)[None, :]]
31     return dict(point_ld=point_ld, point_pv=point_pv,
32                 scen_ld=np.asarray(scen_ld), scen_pv=np.asarray(scen_pv))
33
34
35 def pv_scenarios(point_pv, hist_pv, S=30, seed=1, sigma_scale=1.0):
36     """以官方点预报 point_pv 为中心，叠加历史光伏(逐槽)变异 → 情景(kWh)。

```

```

37 hist_pv: (N,144)。"""
38 rng = np.random.default_rng(seed)
39 T = C.T_IN_DAY
40 n = len(hist_pv)
41 idx = rng.integers(0, max(n, 1), size=(S, T)) if n > 0 else np.zeros((S, T), int)
42 if n > 0:
43     # 历史同槽相对偏差
44     hist = np.maximum(hist_pv, 0.0)
45     base = np.tile(point_pv, (S, 1))
46     offset = hist[idx, np.arange(T)[None, :]] - point_forecast(hist_pv, n)
47     scen = base + sigma_scale * offset
48     scen = np.maximum(scen, 0.0)
49 else:
50     scen = np.tile(point_pv, (S, 1))
51 return np.asarray(scen, float)
52
53
54 def residual_scenarios(load_hist, pv_hist, K, S=30, seed=1):
55     """整日残差 bootstrap 情景生成（升级版：负荷与光伏同一天成对抽取，替代逐槽独立抽样）。
56
57     load_hist/pv_hist: (N,144) 按日期升序的历史实际值(kWh/时段)。
58     调用方必须只传入目标日 d 之前的数据（信息因果，禁止含未来）。
59     K: int（负荷/光伏同一窗口）或 (K_L, K_P)（各自窗口长度）。
60     点预报 = 各自最近 K 天逐槽均值；
61     残差池 = 池内每一天的（实际 - 点预报）整条曲线（K_L=K_P 时即最近 K 天）；
62     抽样 = 有放回抽 S 个整天，负荷与光伏使用同一天索引（成对，保留负荷-光伏相关性）；
63     情景 = 点预报 + 整日残差；光伏情景截断到 >=0（物理非负）。
64     返回 dict(point_ld, point_pv, scen_ld, scen_pv), scen_*(S,144)。
65     """
66     if isinstance(K, (tuple, list, np.ndarray)):
67         KL, KP = int(K[0]), int(K[1])
68     else:
69         KL = KP = int(K)
70     T = C.T_IN_DAY
71     n = len(load_hist)
72     rng = np.random.default_rng(seed)
73     point_ld = point_forecast(load_hist, KL)
74     point_pv = point_forecast(pv_hist, KP)
75     if n == 0:
76         return dict(point_ld=point_ld, point_pv=point_pv,
77                     scen_ld=np.tile(point_ld, (S, 1)),
78                     scen_pv=np.tile(point_pv, (S, 1)))
79     # 成对抽样池：取并集窗口（最近 max(KL,KP) 天），同一池内日索引同时决定负荷与光伏残差
80     w = min(n, max(KL, KP))
81     resid_ld = load_hist[n - w:n] - point_ld[None, :]
82     resid_pv = pv_hist[n - w:n] - point_pv[None, :]
83     idx = rng.integers(0, w, size=S) # 池内偏移（0=最早一天，w-1=最近一天）

```

```

84     scen_ld = point_ld[None, :] + resid_ld[idx]
85     scen_pv = point_pv[None, :] + resid_pv[idx]
86     scen_pv = np.maximum(scen_pv, 0.0)
87     return dict(point_ld=point_ld, point_pv=point_pv,
88                 scen_ld=np.asarray(scen_ld, float), scen_pv=np.asarray(scen_pv, float))

```

## 2.5 B.5 问题一入口 (main\_q1.py)

单日确定性完美信息；调用 solve\_day 求第 1 问计划购电、充放电与储电轨迹，写出 result1.xlsx。

```

1  # -*- coding: utf-8 -*-
2  """问题1: 确定性 MILP/LP 日计划 (附件1 完美信息)。
3
4  储能日循环: storage 日末回到日初(6000 kWh), 保证可长期重复运行;
5  目标: 最小化当日购电费用(电价 p • 购电量), 约束含 能量平衡、储能上下限、充放电功率上限、
6         光伏/负载(附件1 预报)已知。输出 result1.xlsx(两张表) + 计划图。
7  """
8  import os, sys
9  sys.path.insert(0, os.path.dirname(__file__))
10 import numpy as np
11 import pandas as pd
12 import matplotlib
13 matplotlib.use('Agg')
14 import config as C
15 C.setup_plot_style()
16 import matplotlib.pyplot as plt
17 import data, lp, results as R
18
19
20 def run(outfile='result1.xlsx'):
21     a1 = data.load_attach1()
22     price = a1['price']
23     rp = lp.solve_day_plan(price, a1['pv'], a1['load'], C.E_INIT, force_end=C.E_INIT)
24     P, c, f, E = rp['P'], rp['c'], rp['f'], rp['E']
25     cost = float(np.sum(price * P))
26     path = os.path.join(C.RES_DIR, outfile)
27     R.write_result1(path, P, c, f, C.E_INIT, E[-1], price)
28     print('已写出', path)
29     print(f'[Q1 确定性日计划] 购电量合计={P.sum():.2f} kWh, 购电费={cost:.2f} 元, '
30           f'0:00 储电={C.E_INIT}, 24:00 储电={E[-1]:.2f}')
31     return dict(price=price, P=P, c=c, f=f, E=E, cost=cost)
32
33

```



```

34 def plot(P, c, f, E, price, path=None):
35     """计划购电/充电/放电/储电量/电价 可视化。"""
36     x = np.arange(C.T_IN_DAY)
37     fig, ax1 = plt.subplots(figsize=(13, 5.5))
38     ax1.bar(x, P, width=0.6, alpha=0.7, label='购电量(kWh)', color='tab:blue')
39     ax1.bar(x, c, width=0.6, alpha=0.5, label='充电量(kWh)', color='tab:green')
40     ax1.bar(x, f, width=0.6, alpha=0.5, label='放电量(kWh)', color='tab:orange')
41     ax1.set_xlabel('时段(每10min)', fontsize=13); ax1.set_ylabel('能量(kWh)', fontsize=13)
42     ax1.set_xlim(0, C.T_IN_DAY)
43     ax1.set_xticks(range(0, C.T_IN_DAY + 1, 12))
44     ax1.legend(loc='upper left', fontsize=12); ax1.grid(alpha=0.3)
45     ax1.tick_params(labelsize=11)
46
47     ax2 = ax1.twinx()
48     ax2.plot(x, E, color='tab:red', lw=1.6, label='储电量(kWh)')
49     ax2.axhline(C.SOC_HIGH_BOUND, color='gray', ls=':', lw=1)
50     ax2.axhline(C.SOC_MIN, color='gray', ls=':', lw=1)
51     ax2.set_ylabel('储电量(kWh)', fontsize=13); ax2.legend(loc='upper right', fontsize=12)
52     ax2.set_ylim(0, C.SOC_MAX)
53     ax2.tick_params(labelsize=11)
54     plt.title('问题1: 单日最优运营计划 (储能日循环)', fontsize=14)
55     fig.tight_layout()
56     path = path or os.path.join(C.FIG_DIR, 'q1_plan.png')
57     fig.savefig(path, dpi=150); plt.close(fig)
58     print('图已存', path)
59
60
61 def dp_cost_day(price, G, D, E_start=C.E_INIT, E_end=C.E_INIT, h=120.0):
62     """状态空间动态规划精确求解单日最小购电费 (网格步长 h, 验证 LP 全局最优性)。
63
64     状态 = 时段末储电量  $E \in \{1200, 1200+h, \dots, 10800\}$ ;
65     转移  $E \rightarrow E'$ : 若  $\Delta = E' - E > 0$  则  $c = \Delta / \eta$ 、 $f = 0$ ;  $\Delta < 0$  则  $f = -\Delta$ 、 $c = 0$ ;
66     购电  $P = \max(D - G - \eta \cdot f + c, 0)$  (弃光免费);  $V_t(E) = \min_{\{E'\}}(p_t \cdot P + V_{t+1}(E'))$ ,
67     终端  $V_{\{144\}}(E_{\text{end}}) = 0$ , 起点  $V_0(E_{\text{start}})$  即最优费用。向量化: 每时段对
68      $|\Delta| \leq \min(E_{\text{FMAX}}, \eta \cdot E_{\text{CMAX}})$  的网格位移求 min。
69     """
70     T = C.T_IN_DAY
71     grid = np.arange(C.SOC_MIN, C.SOC_HIGH_BOUND + 1e-9, h)
72     n = len(grid)
73     idx = {round(float(e), 6): i for i, e in enumerate(grid)}
74     if round(float(E_start), 6) not in idx or round(float(E_end), 6) not in idx:
75         raise ValueError('起/止储电量不在 DP 网格上 (请选网格 h 整除 1200..10800)')
76     K = int(np.floor(min(C.E_FMAX, C.ETA * C.E_CMAX) / h))
77     ks = np.arange(-K, K + 1)
78     dE = ks * h
79     f = np.maximum(-dE, 0.0) # 放电量
80     c = np.maximum(dE / C.ETA, 0.0) # 充电量

```

```

81 Pk = np.maximum(D[:, None] - G[:, None] - C.ETA * f[None, :] + c[None, :], 0.0) # (T, 2K+1)
82 INF = np.inf
83 V = np.full(n, INF)
84 V[idx[round(float(E_end), 6)]] = 0.0
85 for t in range(T - 1, -1, -1):
86     cand = np.full((n, 2 * K + 1), INF)
87     for k2, k in enumerate(ks):
88         j = np.arange(n) + k
89         ok = (j >= 0) & (j < n)
90         cand[ok, k2] = price[t] * Pk[t, k2] + V[j[ok]]
91     V = cand.min(axis=1)
92 return float(V[idx[round(float(E_start), 6)]])
93
94
95 def dp_report(price=None):
96     """问题1 LP 最优性动态规划验证。
97
98     (1) 附件1 典型日: 网格  $h \in \{120, 60, 30, 15\}$  的 DP 费用收敛到 LP 解 ( $h=15$  偏差  $\leq 0.5\%$ ),
99     验证 LP 全局最优;
100    (2) 附件2 全年逐日:  $h=15$  下 DP 与 LP (均  $E_0=E_{144}=6000$  日循环) 费用差,
101    报告最大/平均相对偏差 (网格离散化误差)。
102    写 report/q1_dp_verification.csv 与 paper/figures_adv/A13_q1_dp.png。
103    """
104    a1 = data.load_attach1()
105    price = a1['price'] if price is None else price
106    G, D = a1['pv'], a1['load']
107    rp = lp.solve_day_plan(price, G, D, C.E_INIT, force_end=C.E_INIT)
108    lp_cost = float(np.sum(price * rp['P']))
109
110    rows = []
111    print(f'[Q1 DP 验证] 附件1 典型日: 网格收敛 (LP 解 = {lp_cost:,.2f} 元)')
112    conv = {}
113    for h in (120.0, 60.0, 30.0, 15.0):
114        dp = dp_cost_day(price, G, D, h=h)
115        conv[h] = dp
116        rows.append(dict(day='附件1典型日', grid_h=h, lp_cost=lp_cost,
117                        dp_cost=dp, gap_pct=(dp - lp_cost) / lp_cost * 100))
118        print(f'h={h:g}: DP={dp:,.2f} 元 gap={(dp - lp_cost) / lp_cost * 100:.4f}%')
119
120    dates, load, pv = data.load_attach2()
121    gaps = []
122    for i in range(len(dates)):
123        rpi = lp.solve_day_plan(price, pv[i], load[i], C.E_INIT, force_end=C.E_INIT)
124        lpi = float(np.sum(price * rpi['P']))
125        dpi = dp_cost_day(price, pv[i], load[i], h=15.0)
126        gaps.append(dpi - lpi)
127        rows.append(dict(day=str(dates[i].date()), grid_h=15, lp_cost=round(lpi, 4),

```

```

128         dp_cost=round(dpi, 4), gap_pct=(dpi - lpi) / lpi * 100))
129     gaps = np.asarray(gaps)
130     gap_p = gaps / np.asarray([r['lp_cost'] for r in rows if r['day'] != '附件1典型日']) * 100
131     print(f'[Q1 DP 验证] 附件2 全年 {len(dates)} 日 (h=15): max gap = {gaps.max():.2f} 元 '
132           f'({gap_p.max():.3f}%), mean gap = {gaps.mean():.2f} 元 ({gap_p.mean():.3f}%)')
133     path = os.path.join(C.RES_DIR, 'q1_dp_verification.csv')
134     pd.DataFrame(rows).to_csv(path, index=False, encoding='utf-8-sig')
135     print('已写', path)
136
137     # A13 图: 左=网格收敛, 右=全年逐日偏差分布
138     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.3))
139     hs = sorted(conv)
140     ax1.plot(hs, [conv[h] for h in hs], '-o', color='tab:blue')
141     ax1.axhline(lp_cost, color='tab:red', ls='--', lw=1.2, label=f'LP 最优 = {lp_cost:,.2f} 元')
142     ax1.set_xscale('log', base=2); ax1.invert_axis()
143     ax1.set_xticks(list(hs)); ax1.set_xticklabels([f'{h:g}' for h in hs], fontsize=12)
144     ax1.set_xlabel('DP 网格步长 h (kWh)', fontsize=13)
145     ax1.set_ylabel('最小购电费 (元)', fontsize=13)
146     ax1.set_title('(a) 附件1 典型日: DP 网格收敛', fontsize=14)
147     ax1.legend(fontsize=12); ax1.grid(alpha=0.3)
148     ax1.tick_params(labelsize=11)
149     gp = np.asarray([r['gap_pct'] for r in rows if r['day'] != '附件1典型日'])
150     ax2.hist(gp, bins=30, color='tab:green', alpha=0.75, edgecolor='w')
151     ax2.axvline(gp.mean(), color='k', ls='--', lw=1.2, label=f'平均 = {gp.mean():.3f}%')
152     ax2.set_xlabel('DP 相对 LP 偏差 (%)', fontsize=13)
153     ax2.set_ylabel('天数', fontsize=13)
154     ax2.set_title('(b) 附件2 全年逐日偏差分布 (h=15)', fontsize=14)
155     ax2.legend(fontsize=12); ax2.grid(alpha=0.3, axis='y')
156     ax2.tick_params(labelsize=11)
157     fig.tight_layout()
158     out = os.path.join(C.C_BASE, 'paper', 'figures_adv', 'A13_q1_dp.png')
159     os.makedirs(os.path.dirname(out), exist_ok=True)
160     fig.savefig(out, dpi=150); plt.close(fig)
161     print('图已存', out)
162     return rows
163
164
165 if __name__ == '__main__':
166     A = run()
167     plot(A['P'], A['c'], A['f'], A['E'], A['price'])
168     dp_report(A['price'])

```

## 2.6 B.6 问题二入口（main\_q2.py）

按”完美信息 + 滚动”双口径评估两阶段随机 / CVaR 策略，输出 result2.xlsx 与风险敏感性结果。

```
1  # -*- coding: utf-8 -*-
2  """问题2：两阶段随机规划（官方口径 v2 —— 预报式，替代原完美信息口径）。
3
4  官方结果 result2.xlsx 口径：
5      每天 0:00 依据截至前一日的历史数据（信息因果，不含未来）——
6      1) 点预报：负荷/光伏分别取自前 K_L / K_P 天逐槽(144 时段)均值，K 由全年交叉验证选出；
7      2) 情景生成：整日残差 bootstrap (scenarios.residual_scenarios)，负荷与光伏同一天成对，
8          S=30 个情景，抽样种子 = 2026 + 日序（与日期绑定、确定性、重跑逐位一致）；
9          2025-01-01 无历史，按拍板规格用附件1 典型日剖面做冷启动情景基准；
10     3) 计划 P: lp.solve_two_stage(lam=0, 纯期望最小化)；
11     4) 结算：固定 P, 用当日实际负荷/光伏调 lp.solve_day_fixedP（紧急价 5p）得当日实际
12         紧急购电 Q 与充放电 c/f、储能 E; E_end 结转次日（跨日连续，1200~10800 kWh）。
13     全年链自 2025-01-01 (E=6000) 逐日滚动，仅 1 月作储能状态预热，输出 2.1-12.31。
14
15 旧口径：perfect_info_legacy()（把当日实际负荷/光伏当作 0:00 已知 → 全年 Q≡0），
16 仅作对比保留，不再写入 result2.xlsx。
17
18 import os, sys, time, hashlib
19 sys.path.insert(0, os.path.dirname(__file__))
20 import numpy as np
21 import pandas as pd
22 import config as C
23 import data, lp, results as R
24 import scenarios
25
26 K_CANDIDATES = [7, 15, 30] # K 候选集（数据驱动选优）
27 SEED_BASE = 2026           # 抽样种子基数：2026 + 日序
28 LEGACY_FEE_RECORD = 12259844.62 # 旧完美信息口径计划费（历史运行记录值）
29
30
31 def _seed_for_day(d):
32     """与日期绑定的确定性随机种子（重跑逐位一致）。"""
33     return SEED_BASE + int(d.dayofyear)
34
35
36 def cross_validate_K(dates, load, pv):
37     """交叉验证选 K（候选 {7,15,30}）。
38
39     对 2025-02-01~12-31 每天 d: 用 d 前 K 天逐槽均值预报 d, 计算 144 时段 RMSE,
40     除以该日实际均值归一化（当日均值≈0 的天跳过），全年取平均；
41     负荷、光伏各自选出平均归一化 RMSE 最小的 K（记为 K_L、K_P）。"""
```

```

42  返回 (KL, KP, cv 表)。
43  """
44  start = int(np.searchsorted(dates, pd.Timestamp('2025-02-01')))
45  rows, skip = [], 0
46  for K in K_CANDIDATES:
47      nrmse_L, nrmse_P = [], []
48      for i in range(start, len(dates)):
49          lo = max(0, i - K)
50          fcL = load[lo:i].mean(axis=0) if i > lo else load[i] * 0.0
51          fcP = pv[lo:i].mean(axis=0) if i > lo else pv[i] * 0.0
52          mL = float(load[i].mean()); mP = float(pv[i].mean())
53          if mL > 1e-6:
54              nrmse_L.append(float(np.sqrt(np.mean((fcL - load[i]) ** 2)) / mL))
55          else:
56              skip += 1
57          if mP > 1e-6:
58              nrmse_P.append(float(np.sqrt(np.mean((fcP - pv[i]) ** 2)) / mP))
59          else:
60              skip += 1
61      rows.append(dict(K=K, nrmse_load=float(np.mean(nrmse_L)),
62                      nrmse_pv=float(np.mean(nrmse_P))))
63  cv = pd.DataFrame(rows)
64  KL = int(cv.loc[cv['nrmse_load'].idxmin(), 'K'])
65  KP = int(cv.loc[cv['nrmse_pv'].idxmin(), 'K'])
66  if skip:
67      print(f'[K 交叉验证] 跳过日均值≈0 的天次（负荷/光伏合计）: {skip}')
68  return KL, KP, cv
69
70
71 def perfect_info_legacy(price, load, pv, dates):
72     """【旧口径，仅作对比】完美信息：把当日实际负荷/光伏当作 0:00 已知，
73     确定性 LP 计划 → 全年紧急购电 Q≡0。不再用于官方 result2.xlsx。"""
74     rec = {}
75     E_start = C.E_INIT
76     for i in range(len(dates)):
77         rp = lp.solve_day_plan(price, pv[i], load[i], E_start)
78         rec[dates[i]] = dict(P=rp['P'], Q=np.zeros(C.T_IN_DAY), c=rp['c'], f=rp['f'],
79                             E_start=E_start, E_end=rp['E'][-1])
80         E_start = rp['E'][-1]
81     return rec
82
83
84 def _run_forecast_chain(price, dates, load, pv, KL, KP, S=30, lam=0.0,
85                         max_days=None, verbose=True):
86     """预报式两阶段随机规划全年滚动链（官方口径）。
87
88     每日: residual_scenarios(load[:i], pv[:i], (KL,KP), S, seed=2026+日序)

```

```

89 → solve_two_stage(lam) 得计划 P → solve_day_fixedP 按当日实际结算
90 → E_end 结转次日。断言每日储能 E 全程 ∈ [1200,10800]。
91 max_days: 仅跑前 N 天 (冒烟测试用), 默认全年。
92 """
93 n = len(dates) if max_days is None else int(max_days)
94 rec = {}
95 E_start = C.E_INIT
96 t0 = time.time()
97 a1 = data.load_attach1()      # 冷启动先验: 2025-01-01 无历史可用
98 for i in range(n):
99     d = dates[i]
100     # price: 一维固定电价(144) 或 二维逐日电价(365,144) (Q4-2 波动电价用)
101     p = price[i] if price.ndim == 2 else price
102     if i == 0:
103         # 冷启动 (拍板规格): 用附件1 典型日负荷/光伏剖面作当日点预报与情景基准
104         # (无历史可抽残差, S 个情景同形)。
105         scen_ld = np.tile(a1['load'], (S, 1))
106         scen_pv = np.tile(a1['pv'], (S, 1))
107     else:
108         fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=S,
109                                           seed=_seed_for_day(d))
110         scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
111     two = lp.solve_two_stage(p, scen_ld, scen_pv, E_start,
112                             lam=lam, beta=C.BETA)
113     settle = lp.solve_day_fixedP(p, pv[i], load[i], E_start, two['P'])
114     E_t = np.asarray(settle['E'], float)
115     assert np.all(E_t >= C.SOC_MIN - 1e-6) and np.all(E_t <= C.SOC_HIGH_BOUND + 1e-6), \
116         f'{d.date()} 储能越界: min={E_t.min():.2f} max={E_t.max():.2f}'
117     rec[d] = dict(P=two['P'], Q=settle['Q'], c=settle['c'], f=settle['f'],
118                  E_start=float(E_start), E_end=float(E_t[-1]))
119     E_start = float(E_t[-1])
120     if verbose and (i + 1) % 30 == 0:
121         e1 = time.time() - t0
122         print(f' 滚动链进度 {i + 1}/{n} 日 ({d.date()}) E={E_start:8.1f} kWh'
123              f' 用时 {e1:6.1f}s', flush=True)
124 return rec
125
126
127 def yearly_lambda_scan(price, dates, load, pv, KL, KP, lams=(0.0, 0.5, 1.0, 2.0, 5.0),
128                       S=30, max_days=None):
129     """全年 λ 扫描: 每个 λ 自 1-01 整链重跑 (1 月预热), 汇总全年费用。
130
131     用于数据驱动选定官方口径 λ*: emerg(λ) ≤ 0.6 · emerg(0) 且 total(λ) ≤ 1.01 · total(0)
132     时认为风险权衡"以可忽略的总费上升换取紧急购电显著削减", 取满足条件的最小 λ。
133     返回 (df, λ*), df 写 report/q2_lambda_year_scan.csv。
134     """
135     rows = []

```

```

136 for lam in lams:
137     rec = _run_forecast_chain(price, dates, load, pv, KL, KP, S=S, lam=float(lam),
138                             max_days=max_days, verbose=False)
139     d_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
140     pf = sum(float(np.sum(price * rec[d]['P']))) for d in d_out)
141     ef = sum(float(np.sum(C.EMERG_MULT * price * rec[d]['Q']))) for d in d_out)
142     ek = sum(float(rec[d]['Q'].sum())) for d in d_out)
143     rows.append(dict(lam=float(lam), plan_fee=round(pf, 2), emerg_fee=round(ef, 2),
144                    total=round(pf + ef, 2), emerg_kwh=round(ek, 2)))
145     print(f'[λ 扫描] λ={lam}: 计划={pf:,.2f} 紧急={ef:,.2f} ({ek:,.0f} kWh) '
146           f' 合计={pf + ef:,.2f}')
147 df = pd.DataFrame(rows)
148 path = os.path.join(C.RES_DIR, 'q2_lambda_year_scan.csv')
149 df.to_csv(path, index=False, encoding='utf-8-sig')
150 print('已写', path)
151 # 数据驱动选 λ*: emerg 下降 ≥40% 且 total 不升超 1%
152 base = df[df.lam == lams[0]].iloc[0]
153 cand = df[(df.emerg_fee <= 0.6 * base.emerg_fee) & (df.total <= 1.01 * base.total)]
154 lam_star = float(cand.lam.min()) if len(cand) else float(lams[0])
155 print(f'[λ* 选定] λ* = {lam_star} (基准 emerg={base.emerg_fee:,.2f},
156        total={base.total:,.2f}) ')
157 return df, lam_star
158
159 def _result_digest(price, dates_out, rec):
160     """对全部输出序列做确定性哈希，供跨次运行逐位一致性校验。"""
161     h = hashlib.sha256()
162     for d in dates_out:
163         h.update(np.round(np.asarray(rec[d]['P'], float), 6).tobytes())
164         h.update(np.round(np.asarray(rec[d]['Q'], float), 6).tobytes())
165         h.update(np.round(np.asarray(rec[d]['C'], float), 6).tobytes())
166         h.update(np.round(np.asarray(rec[d]['f'], float), 6).tobytes())
167         h.update(np.round(np.float64(rec[d]['E_end'])), 6).tobytes())
168     return h.hexdigest()
169
170
171 def run(outfile='result2.xlsx', lam=None, S=None, max_days=None):
172     a1 = data.load_attach1()
173     price = a1['price']
174     dates, load, pv = data.load_attach2()
175     S = C.S_NUM if S is None else int(S)
176
177     # ---- 1) 交叉验证选 K (数据驱动) ----
178     KL, KP, cv = cross_validate_K(dates, load, pv)
179     print('[K 交叉验证] 全年平均归一化 RMSE (2025-02-01~12-31, 144 时段):')
180     print(cv.to_string(index=False))
181     print(f'所选: K_L = {KL}, K_P = {KP}')

```

```

182
183 # ---- 1b)  $\lambda^*$  全年扫描（数据驱动选官方风险权衡权重） ----
184 if lam is None:
185     scan, lam_star = yearly_lambda_scan(price, dates, load, pv, KL, KP,
186                                         S=S, max_days=max_days)
187     lam = lam_star
188     print(f'[Q2 官方口径] 采用数据驱动选定  $\lambda^* = \{lam\_star\}$ ')
189 else:
190     scan = None
191     lam = float(lam)
192
193 # ---- 2) 预报式两阶段随机规划全年链（1 月预热，不输出） ----
194 print(f'开始全年滚动链: 365 天  $\times S=\{S\}$  情景两阶段 LP ( $lam=\{lam\}$ ) .....')
195 rec = _run_forecast_chain(price, dates, load, pv, KL, KP, S=S, lam=lam,
196                           max_days=max_days)
197
198 # ---- 3) 汇总与写 result2.xlsx ----
199 dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
200 P = {d: rec[d]['P'] for d in dates_out}
201 Q = {d: rec[d]['Q'] for d in dates_out}
202 c = {d: rec[d]['c'] for d in dates_out}
203 f = {d: rec[d]['f'] for d in dates_out}
204 E0 = {d: rec[d]['E_start'] for d in dates_out}
205 Ee = {d: rec[d]['E_end'] for d in dates_out}
206 plan_fee = sum(float(np.sum(price * rec[d]['P'])) for d in dates_out)
207 emerg_fee = sum(float(np.sum(C.EMERG_MULT * price * rec[d]['Q'])) for d in dates_out)
208 emerg_kwh = sum(float(rec[d]['Q'].sum()) for d in dates_out)
209 total_fee = plan_fee + emerg_fee
210 assert emerg_fee > 0, '全年紧急购电费应 > 0（旧完美信息口径为 0，已切换预报式口径）'
211 path = os.path.join(C.RES_DIR, outfile)
212 R.write_result2(path, dates_out, P, Q, c, f, E0, Ee)
213 path = R.strip_personal_meta(path) # 清除模板带入的个人元数据（提交合规）
214 print('已写出', path)
215 print(f'[Q2 官方·预报式] 计划购电费 = {plan_fee:,.2f} 元 | '
216       f'紧急购电费 = {emerg_fee:,.2f} 元（{emerg_kwh:,.0f} kWh） | '
217       f'合计 = {total_fee:,.2f} 元')
218
219 # ---- 4) 指定日期 144 时段 Q 序列（论文表3 用） ----
220 spec_q = {}
221 print('[Q2 指定日期紧急购电（论文表3）]')
222 for ds in C.SPEC_DAYS:
223     d = pd.Timestamp(ds)
224     spec_q[d] = np.asarray(rec[d]['Q'], float)
225     if spec_q[d].sum() <= 0:
226         # 合法零值：当日实际净需求低于全部情景（如周末低负荷日），计划按情景包络
227         # 多购，缺电由计划+储能完全吸收，表现为高弃光而非紧急购电。
228         print(f'注意：{ds} 紧急购电 = 0 kWh（合法结果：当日实际净需求低于全部 S 个'

```



```

229         f'情景，计划按情景包络多购，缺电由计划+储能吸收，弃光较高')
230     else:
231         print(f' {ds}: 紧急购电 {spec_q[d].sum():.1f} kWh, 紧急费 '
232               f'{float(np.sum(C.EMERG_MULT * price * spec_q[d])):.1f} 元')
233     qtab = pd.DataFrame({'时间段': C.INTERVAL_LABELS})
234     for ds in C.SPEC_DAYS:
235         qtab[ds] = np.round(spec_q[pd.Timestamp(ds)], 4)
236     qtab.to_csv(os.path.join(C.RES_DIR, 'q2_specdays_Q.csv'),
237                 index=False, encoding='utf-8-sig')
238
239     # ---- 5) 摘要 CSV ----
240     summ_rows = [
241         ('全年计划购电费_元', round(plan_fee, 2)),
242         ('全年紧急购电费_元', round(emerg_fee, 2)),
243         ('合计_元', round(total_fee, 2)),
244         ('全年紧急购电量_kWh', round(emerg_kwh, 2)),
245         ('官方风险权重_λ*', lam),
246         ('K_L', KL), ('K_P', KP),
247     ]
248     for ds in C.SPEC_DAYS:
249         d = pd.Timestamp(ds)
250         summ_rows.append((f'{ds}_紧急购电量_kWh', round(float(spec_q[d].sum()), 4)))
251         summ_rows.append((f'{ds}_紧急购电费_元',
252                           round(float(np.sum(C.EMERG_MULT * price * spec_q[d])), 4)))
253     summ = pd.DataFrame(summ_rows, columns=['指标', '数值'])
254     summ.to_csv(os.path.join(C.RES_DIR, 'q2_new_summary.csv'),
255                 index=False, encoding='utf-8-sig')
256     cv.to_csv(os.path.join(C.RES_DIR, 'q2_K_cv.csv'), index=False, encoding='utf-8-sig')
257
258     # ---- 5b) 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月) ----
259     drow = []
260     for d in dates:
261         dd = rec[d]
262         pf = float(np.sum(price * dd['P']))
263         ef = float(np.sum(C.EMERG_MULT * price * dd['Q']))
264         drow.append(dict(date=str(d.date()), plan_fee=round(pf, 2),
265                          emerg_fee=round(ef, 2), total=round(pf + ef, 2)))
266     pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q2_daily_cost.csv'),
267                              index=False, encoding='utf-8-sig')
268
269     # ---- 6) 重跑一致性校验 (seed 固定 → 逐位一致; 口径变更需删旧 digest) ----
270     digest_path = os.path.join(C.RES_DIR, 'q2_digest.txt')
271     dg = _result_digest(price, dates_out, rec)
272     if os.path.exists(digest_path):
273         old = open(digest_path, encoding='utf-8').read().strip()
274         if old != dg:
275             print(f'[重跑一致性] digest 变化 (口径/λ 变更所致): 旧={old[:16]}... '

```

```

276         f'新={dg[:16]}...', 已记录新 digest')
277     with open(digest_path, 'w', encoding='utf-8') as fp:
278         fp.write(dg)
279     else:
280         print(f'[重跑一致性] digest 校验通过: {dg[:16]}...')
281 else:
282     with open(digest_path, 'w', encoding='utf-8') as fp:
283         fp.write(dg)
284     print(f'[重跑一致性] 首次运行, 已记录 digest: {dg[:16]}...')
285
286 # ---- 7) 与旧完美信息口径对比 (仅对比, 不写结果) ----
287 leg = perfect_info_legacy(price, load, pv, dates)
288 leg_fee = sum(float(np.sum(price * leg[d]['P']))) for d in dates_out)
289 diff = plan_fee - leg_fee
290 print('\n[新旧口径对比]')
291 print(f'旧完美信息计划费 = {leg_fee:,.2f} 元'
292       f' (历史记录 {LEGACY_FEE_RECORD:,.2f} 元, '
293       f'{"复算吻合" if abs(leg_fee - LEGACY_FEE_RECORD) < 1.0 else "复算不一致, 请检查"}) ')
294 print(f'新预报表式计划费 = {plan_fee:,.2f} 元, 差 = {diff:+,.2f} 元')
295 if diff > 0:
296     print('解释: 完美信息口径下计划即当日实际最优购电 (全年紧急购电恒为 0), 是计划费的'
297           '下界; 预报表式口径下计划基于历史均值与整日残差情景做期望最小化, 为对冲负荷/光伏'
298           '不确定性会在低价时段多购电充当“保险”, 把缺电风险尽量转移到 1 倍计划电价而非 '
299           '5 倍紧急电价, 因此计划费略高 (保险性多购), 多出的计划费换来的是把实际缺电'
300           f'风险真实计量为全年 {emerg_fee:,.0f} 元紧急购电费 (旧口径 Q=0, 该风险成本'
301           '为零计量)。')
302 else:
303     print('解释: 预报表式口径计划费不高于完美信息口径, 说明历史均值预报在低价时段购电更'
304           '克制; 完美信息计划因精确贴合当日曲线而把更多购电安排在低价时段, 两者均无紧急'
305           '购电的完美信息是理论下界, 预报表式口径的全部费用 = 计划费 + 紧急费才是真实可'
306           '支付成本。')
307 print(f'[Q2 完成] 全年合计费用 (计划+紧急) = {total_fee:,.2f} 元')
308 return dict(price=price, dates=dates, load=load, pv=pv, rec=rec,
309            dates_out=dates_out, KL=KL, KP=KP, lam=lam,
310            plan_fee=plan_fee, emerg_fee=emerg_fee, total_fee=total_fee,
311            leg_fee=leg_fee)
312
313
314 def method_analysis(price, dates, load, pv, rec, KL, KP, lam_star=0.0):
315     """  $\lambda$  (CVaR 权重) 敏感性 (论文用, 不计入 result2.xlsx) 。
316
317     4 个指定日期  $\times \lambda \in \{0, 0.5, 1, 2, 5\}$ : 新情景生成器 ((K_L, K_P) 整日成对残差 bootstrap,
318     seed=2026+日序, 与官方链同源)  $\rightarrow$  solve_two_stage(lam)  $\rightarrow$  当日实际结算。
319     输出 plan_fee / emerg_fee / total / emerg_kwh 表 (CSV) 并画 3.20 权衡图。
320     """
321     rows = []
322     for dd in C.SPEC_DAYS:

```

```

323     d = pd.Timestamp(dd)
324     i = int(np.where(dates == d)[0][0])
325     E_start = rec[d]['E_start']
326     fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=C.S_NUM,
327                                     seed=_seed_for_day(d))
328     for lam in [0.0, 0.5, 1.0, 2.0, 5.0]:
329         two = lp.solve_two_stage(price, fc['scen_ld'], fc['scen_pv'], E_start,
330                                 lam=lam, beta=C.BETA)
331         settle = lp.solve_day_fixedP(price, pv[i], load[i], E_start, two['P'])
332         plan_fee = float(np.sum(price * two['P']))
333         emerg_fee = float(np.sum(C.EMERG_MULT * price * settle['Q']))
334         rows.append(dict(day=dd, lam=lam, plan_fee=plan_fee, emerg_fee=emerg_fee,
335                         total=plan_fee + emerg_fee, emerg_kwh=float(settle['Q'].sum())))
336 df = pd.DataFrame(rows)
337 # 自检:  $\lambda = \lambda \star$  行应与官方链当日结算一致 (同种子、同情景、同 E_start)
338 for dd in C.SPEC_DAYS:
339     d = pd.Timestamp(dd)
340     r0 = df[(df.day == dd) & (df.lam == lam_star)].iloc[0]
341     chain_plan = float(np.sum(price * rec[d]['P']))
342     chain_emer = float(np.sum(C.EMERG_MULT * price * rec[d]['Q']))
343     assert abs(r0.plan_fee - chain_plan) < 1e-3, f'{dd}  $\lambda = \{lam\_star\}$  计划费与官方链不一致'
344     assert abs(r0.emerg_fee - chain_emer) < 1e-3, f'{dd}  $\lambda = \{lam\_star\}$  紧急费与官方链不一致'
345 print('\n[Q2  $\lambda$  敏感性 • 两阶段随机+CVaR (新情景生成器)]')
346 print(df.to_string(index=False))
347 df.to_csv(os.path.join(C.RES_DIR, 'q2_two_stage_comparison.csv'),
348           index=False, encoding='utf-8-sig')
349
350 import matplotlib
351 matplotlib.use('Agg')
352 C.setup_plot_style()
353 import matplotlib.pyplot as plt
354 df20 = df[df.day == C.SPEC_DAYS[0]].sort_values('lam')
355 fig, ax = plt.subplots(figsize=(8, 5))
356 ax.plot(df20.lam, df20.plan_fee, '-o', label='计划购电费')
357 ax.plot(df20.lam, df20.emerg_fee, '-s', label='紧急购电费')
358 ax.plot(df20.lam, df20.total, '--^', label='合计')
359 ax.set_xlabel('CVaR 权重  $\lambda$ '); ax.set_ylabel('费用(元)')
360 ax.set_title(f'问题2 两阶段随机+CVaR: {C.SPEC_DAYS[0]} 风险-费用权衡')
361 ax.legend(); ax.grid(alpha=0.4)
362 fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q2_cvar_tradeoff.png'), dpi=150)
363 plt.close(fig)
364 print('已写 report/q2_two_stage_comparison.csv 与 figures/q2_cvar_tradeoff.png')
365
366
367 if __name__ == '__main__':
368     A = run()
369     method_analysis(A['price'], A['dates'], A['load'], A['pv'], A['rec'], A['KL'], A['KP'],

```

```
lam_star=A['lam']) # 链已按  $\lambda^*$  运行, 自检比对  $\lambda=\lambda^*$  行
```

## 2.7 B.7 问题三入口 (main\_q3.py)

滚动随机模型预测控制: 0:00 计划、6/12/18 调整、实际光伏结算与违约/超额惩罚, 写出 result3.xlsx。

```
1  # -*- coding: utf-8 -*-
2  """问题3: 滚动随机 MPC + 调整惩罚 (18.0 因果严谨版)。
```

3

4 0:00 用 附件1 电价 + 0:00 光伏预报(附件3) + 负荷点预报(历史逐槽均值, K=15)制定计划购电 P;

5 6/12/18 用最新光伏预报在"违约50%/超购1.5倍"目标下做\*\*情景鲁棒滚动再计划\*\* (随机 MPC):

6 以 scenarios.pv\_scenarios 生成 S=30 个光伏情景, 一阶段购电 A 共用、各情景独立电池调度

7 (允许情景内紧急购电 5p 保证可行), 目标 = 期望电网费 (lam=0);

8 调整时点储能用\*\*实际 SOC 回放\*\* (以已承诺购电 A[:t0] + 当日实际负荷/光伏回放至 t0);

9 结算\*\*去后见之明\*\*: 逐时段 t 以 [0..t] 前缀 (固定购电 A[:t+1]、实际负荷/光伏到 t) 重解电池,

10 取末段 c/f/Q/E 结转, 替代"全天实际重优化"。

11 费用口径: cost\_t = p \* min(P,A) + 0.5p \* (P-A)^+ + 1.5p \* (A-P)^+ + 5p \* Q\_t。

12 """

13 import os, sys

14 sys.path.insert(0, os.path.dirname(\_\_file\_\_))

15 import numpy as np

16 import pandas as pd

17 import config as C

18 import data, lp, results as R

19 import scenarios

20 C.setup\_plot\_style()

21 import matplotlib

22 matplotlib.use('Agg')

23 import matplotlib.pyplot as plt

24 TAU = [0, 6, 12, 18]

25 SEED = 2026 # 情景抽样种子基数: 2026 + 日序 (与 Q2 对齐, 确定性可复现)

26 K\_LOAD = 15 # 负荷点预报窗口 (与 Q2 K\_L 交叉验证结论对齐)

27

28

29 def \_load\_fc\_point(hist, K=7):

30 n = len(hist)

31 if n == 0:

32 return np.zeros(C.T\_IN\_DAY)

33 return np.mean(hist[-min(K, n):], axis=0)

34

35

36 def \_adjust(price, P, A, Q):

37 """按费用口径计算: 电网费用 + 紧急购电费用。返回 (grid\_fee, emerg\_fee, fee\_vec)。"""

```

38     u = np.maximum(P - A, 0.0)
39     w = np.maximum(A - P, 0.0)
40     grid_fee = float(np.sum(price * np.minimum(P, A) + C.DEFAULT_MULT * price * u +
41                             C.EXCESS_MULT * price * w))
42     emerg_fee = float(np.sum(C.EMERG_MULT * price * Q))
43     base = price * np.minimum(P, A) + C.DEFAULT_MULT * price * u + C.EXCESS_MULT * price * w +
44           C.EMERG_MULT * price * Q
45     return grid_fee, emerg_fee, base
46
47 def causal_settle(price, pv_act, load_act, E_start, A):
48     """去后见之明结算（替代全天实际重优化）。
49
50     逐时段 t：以 [0..t] 前缀 LP —— 固定购电 A[:t+1]（已承诺）、当日实际负荷/光伏只用到 t ——
51     重解电池充放电，取末段 c/f/Q/R/E 作为 t 时段的真实运行量并结转储能。
52     全程只用 t 及之前的信息，不借用未来实际光伏/负荷。
53     返回 dict(c,f,Q,R,E_end)。
54     """
55     T = C.T_IN_DAY
56     c = np.zeros(T); f = np.zeros(T); Q = np.zeros(T); R = np.zeros(T)
57     E_carry = E_start
58     for t in range(T):
59         res = lp.solve_day(price[:t + 1], pv_act[:t + 1], load_act[:t + 1], E_carry,
60                             T=t + 1, fixed_P=A[:t + 1], allow_P=False, emult=C.EMERG_MULT)
61         c[t] = res['c'][-1]; f[t] = res['f'][-1]
62         Q[t] = res['Q'][-1]; R[t] = res['R'][-1]
63         E_carry = res['E'][-1]
64     return dict(c=c, f=f, Q=Q, R=R, E_end=E_carry)
65
66 def _cold_load():
67     """冷启动负荷预报：2025-01-01 无历史，用附件1 典型日负荷剖面（与 Q2 口径一致）。"""
68     return data.load_attach1()['load']
69
70
71 def _rolling_day(price_day, load_act, pv_act, load_hist, pv_hist, fc3, d, E_start,
72                 K=K_LOAD):
73     """单日滚动核心（18.0 因果严谨版）。
74
75     - 计划/调整用负荷点预报 D_known（load_hist 前 K 天逐槽均值，K=15，与 Q2 对齐；
76       首日无历史时用附件1 典型日剖面冷启动）；
77     - 0:00 计划：solve_day_plan(点预报负荷，0:00 光伏预报)；
78     - 6/12/18 情景鲁棒再计划：pv_scenarios(G_tau, 历史) 生成 S=30 光伏情景，
79       solve_adjust_scen 共用一阶段购电 A；调整时点储能为实际 SOC 回放；
80     - 结算：causal_settle 逐段去后见之明。
81     price_day:(144) 当日电价；load_act/pv_act:(144) 当日实际；load_hist/pv_hist:(i,144)
      截至前一日历史。

```

```

82     返回 dict(...)。
83     """
84     D_known = scenarios.point_forecast(load_hist, K) # 负荷预报化 (历史逐槽均值)
85     if len(load_hist) == 0:
86         D_known = _cold_load() # 冷启动: 附件1 典型日剖面
87     G0 = data.pv_forecast_kwh(fc3, d, 0)[0] # 0:00 光伏预报
88     plan = lp.solve_day_plan(price_day, G0, D_known, E_start)
89     P = plan['P']
90     A = P.copy()
91     for tau in TAU[1:]:
92         t0 = tau * 6
93         G_tau = data.pv_forecast_kwh(fc3, d, tau)[0]
94         # 实际 SOC 回放: 已承诺购电 A[:t0] + 当日实际负荷/光伏至 t0
95         pre = lp.solve_day(price_day[:t0], pv_act[:t0], load_act[:t0], E_start,
96                             T=t0, fixed_P=A[:t0], allow_P=False, emult=C.EMERG_MULT)
97         E_tau = pre['E'][-1]
98         # 情景鲁棒滚动再计划 (随机 MPC): 光伏情景 S=30, 负荷用点预报
99         scenG = scenarios.pv_scenarios(G_tau, pv_hist, S=C.S_NUM,
100                                         seed=SEED + int(d.dayofyear))
101         rp = lp.solve_adjust_scen(price_day, scenG, D_known, E_tau, P, t0, lam=0.0)
102         A[t0:] = rp['A']
103     # 结算去后见之明
104     settle = causal_settle(price_day, pv_act, load_act, E_start, A)
105     grid_fee, emerg_fee, _ = _adjust(price_day, P, A, settle['Q'])
106     return dict(P=P, A=A, Q=settle['Q'], c=settle['c'], f=settle['f'],
107                 grid_fee=grid_fee, emerg_fee=emerg_fee, E_end=settle['E_end'])
108
109
110 def _rolling(price, load, pv, dates, fc3, i, E_start, K=K_LOAD):
111     """单日滚动 (18.0 因果严谨版, 按日期索引取数后委托 _rolling_day)。"""
112     return _rolling_day(price, load[i], pv[i], load[:i], pv[:i], fc3, dates[i],
113                         E_start, K=K)
114
115
116 def run(outfile='result3.xlsx'):
117     a1 = data.load_attach1()
118     price = a1['price']
119     dates, load, pv = data.load_attach2()
120     fc3 = data.load_attach3()
121     rec = {}
122     E_start = C.E_INIT
123     for i in range(len(dates)):
124         d = dates[i]
125         res = _rolling(price, load, pv, dates, fc3, i, E_start)
126         res['E_start'] = E_start
127         rec[d] = res
128         E_start = res['E_end']

```

```

129
130 dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
131 P = {d: rec[d]['P'] for d in dates_out}
132 A = {d: rec[d]['A'] for d in dates_out}
133 Q = {d: rec[d]['Q'] for d in dates_out}
134 c = {d: rec[d]['c'] for d in dates_out}
135 f = {d: rec[d]['f'] for d in dates_out}
136 E0 = {d: rec[d]['E_start'] for d in dates_out}
137 Ee = {d: rec[d]['E_end'] for d in dates_out}
138 path = os.path.join(C.RES_DIR, outfile)
139 R.write_result3(path, dates_out, P, A, Q, c, f, E0, Ee)
140
141 tot_grid = sum(rec[d]['grid_fee'] for d in dates_out)
142 tot_emerg = sum(rec[d]['emerg_fee'] for d in dates_out)
143 tot = tot_grid + tot_emerg
144 print('已写出', path)
145 print(f'[Q3 滚动MPC] 计划+调整电网费={tot_grid:.2f} 元, 紧急购电费={tot_emerg:.2f} 元,
    合计={tot:.2f} 元')
146
147 # 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月)
148 drow = [dict(date=str(d.date()), grid_fee=round(rec[d]['grid_fee'], 2),
149             emerg_fee=round(rec[d]['emerg_fee'], 2),
150             total=round(rec[d]['grid_fee'] + rec[d]['emerg_fee'], 2)) for d in dates]
151 pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q3_daily_cost.csv'),
152                          index=False, encoding='utf-8-sig')
153 return dict(price=price, dates=dates, load=load, pv=pv, fc3=fc3, rec=rec,
154            dates_out=dates_out)
155
156 def analysis(price, dates, load, pv, fc3, rec):
157     """是否需要其他时刻预报: 比较 仅0:00(不调整) vs 0/6/12/18 滚动 (均用因果结算)。"""
158     rows = []
159     for dd in C.SPEC_DAYS:
160         d = pd.Timestamp(dd)
161         i = int(np.where(dates == d)[0][0])
162         E_start = rec[d]['E_start']
163         D_known = scenarios.point_forecast(load[:i], K_LOAD)
164         G0 = data.pv_forecast_kwh(fc3, d, 0)[0]
165         plan = lp.solve_day_plan(price, G0, D_known, E_start)
166         st = causal_settle(price, pv[i], load[i], E_start, plan['P'])
167         g0, e0, _ = _adjust(price, plan['P'], plan['P'], st['Q'])
168         r = _rolling(price, load, pv, dates, fc3, i, E_start)
169         rows.append(dict(day=dd, only0_grid=g0, only0_emerg=e0, only0_total=g0 + e0,
170                        roll_grid=r['grid_fee'], roll_emerg=r['emerg_fee'],
171                        roll_total=r['grid_fee'] + r['emerg_fee']))
172     df = pd.DataFrame(rows)
173     print('\n[Q3 调整时点分析]')

```

```

174     print(df.to_string(index=False))
175     df.to_csv(os.path.join(C.RES_DIR, 'q3_rolling_comparison.csv'), index=False,
176               encoding='utf-8-sig')
177     x = np.arange(len(df))
178     w = 0.32
179     fig, ax = plt.subplots(figsize=(9, 5))
180     ax.bar(x - w / 2, df.only0_total, w, label='仅0:00(不调整)')
181     ax.bar(x + w / 2, df.roll_total, w, label='0/6/12/18 滚动')
182     ax.set_xticks(x); ax.set_xticklabels(df.day, fontsize=12)
183     ax.set_ylabel('总购电费用(元)', fontsize=13); ax.set_title('问题3: 是否引入多时刻预报',
184               fontsize=14)
185     ax.legend(fontsize=12); ax.grid(alpha=0.4, axis='y')
186     ax.tick_params(labelsize=11)
187     fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q3_forecast_time.png'), dpi=150);
188     plt.close(fig)
189
190 if __name__ == '__main__':
191     A = run()
192     analysis(A['price'], A['dates'], A['load'], A['pv'], A['fc3'], A['rec'])

```

## 2.8 B.8 问题四入口（main\_q4.py）

基于附件4波动电价重算问题二、三，输出 result4-2.xlsx / result4-3.xlsx 并做季节对比。

```

1  # -*- coding: utf-8 -*-
2  """问题4: 波动电价(附件4)重算 问题2/问题3 → result4-2.xlsx / result4-3.xlsx + 对比图。
3
4  附件2(负载/光伏实际)、附件4(波动电价) 均为 全年366×144 矩阵，日期必须对齐。
5  口径沿用（18.0 与 Q2/Q3 官方口径对齐）：
6      Q4-2 = 问题2 预报式两阶段随机规划（KL/KP 沿用 Q2 交叉验证、 $\lambda = \lambda *$  沿用 Q2 官方扫描、
7          seed=2026+日序），仅把固定电价(附件1)换成当日波动电价 → 紧急购电不再恒 0；
8      Q4-3 = 问题3 因果滚动随机 MPC（负荷预报化/实际SOC/去后见之明结算/光伏情景鲁棒），
9          按当日波动电价结算（复用 main_q3._rolling_day）。
10 """
11 import os, sys
12 sys.path.insert(0, os.path.dirname(__file__))
13 import numpy as np
14 import pandas as pd
15 import config as C
16 import data, lp, results as R
17 import main_q2 as MQ2
18 from main_q3 import _rolling_day

```



```

19 C.setup_plot_style()
20 import matplotlib
21 matplotlib.use('Agg')
22 import matplotlib.pyplot as plt
23 TAU = [0, 6, 12, 18]
24
25
26 # ----- Q4-2: 波动电价 + 问题2 预报式两阶段随机规划 -----
27 def run_q42():
28     dates4, price4 = data.load_attach4() # price4[365,144]
29     dates, load, pv = data.load_attach2()
30     data.assert_aligned(dates4, dates, '附件4', '附件2')
31     # K 沿用 Q2 交叉验证 (只依赖负荷/光伏, 与电价口径无关)
32     KL, KP, _ = MQ2.cross_validate_K(dates, load, pv)
33     # λ* 沿用 Q2 官方扫描 (风险权衡权重, 与电价口径无关)
34     summ = pd.read_csv(os.path.join(C.RES_DIR, 'q2_new_summary.csv'))
35     lam_star = float(summ.loc[summ['指标'] == '官方风险权重_λ*', '数值'].iloc[0])
36     print(f'[Q4-2] K_L={KL}, K_P={KP}, λ*={lam_star} (沿用 Q2 官方口径)')
37     rec = MQ2._run_forecast_chain(price4, dates, load, pv, KL, KP, S=C.S_NUM,
38                                   lam=lam_star, verbose=False)
39     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
40     P = {d: rec[d]['P'] for d in dates_out}
41     Q = {d: rec[d]['Q'] for d in dates_out}
42     c = {d: rec[d]['c'] for d in dates_out}
43     f = {d: rec[d]['f'] for d in dates_out}
44     E0 = {d: rec[d]['E_start'] for d in dates_out}
45     Ee = {d: rec[d]['E_end'] for d in dates_out}
46     path = os.path.join(C.RES_DIR, 'result4-2.xlsx')
47     R.write_result2(path, dates_out, P, Q, c, f, E0, Ee, tpl_name='result4-2.xlsx')
48     idx_of = {d: i for i, d in enumerate(dates)}
49     cost = sum(float(np.sum(price4[idx_of[d]] * rec[d]['P'])) for d in dates_out)
50     emerg = sum(float(np.sum(C.EMERG_MULT * price4[idx_of[d]] * rec[d]['Q'])) for d in
51                 dates_out)
52     print('已写出', path)
53     print(f'[Q4-2 波动电价+预报式] 计划费={cost:,.2f} 紧急费={emerg:,.2f} '
54           f'合计={cost + emerg:,.2f} 元, 紧急购电 ≠ 0 kWh')
55     pd.DataFrame({'指标': ['全年计划购电费_元', '全年紧急购电费_元', '合计_元', '紧急购电量_kWh'],
56                  '数值': [round(cost, 2), round(emerg, 2), round(cost + emerg, 2),
57                           round(sum(float(rec[d]['Q']).sum() for d in dates_out), 2)]})
58     .to_csv(os.path.join(C.RES_DIR, 'q4_2_summary.csv'),
59             index=False, encoding='utf-8-sig')
60     # 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月)
61     drow = []
62     for d in dates:
63         dd = rec[d]
64         pf = float(np.sum(price4[idx_of[d]] * dd['P']))
65         ef = float(np.sum(C.EMERG_MULT * price4[idx_of[d]] * dd['Q']))

```

```

65         drow.append(dict(date=str(d.date()), plan_fee=round(pf, 2),
66                         emerg_fee=round(ef, 2), total=round(pf + ef, 2)))
67     pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q4_2_daily_cost.csv'),
68                             index=False, encoding='utf-8-sig')
69     return dict(dates=dates, load=load, pv=pv, price4=price4,
70               rec=rec, dates_out=dates_out, cost=cost, emerg=emerg)
71
72
73     # ----- Q4-3: 波动电价 + 因果滚动随机 MPC -----
74     def _adjust(price, P, A, Q):
75         u = np.maximum(P - A, 0.0)
76         w = np.maximum(A - P, 0.0)
77         grid_fee = float(np.sum(price * np.minimum(P, A) + C.DEFAULT_MULT * price * u +
78                                C.EXCESS_MULT * price * w))
79         emerg_fee = float(np.sum(C.EMERG_MULT * price * Q))
80         return grid_fee, emerg_fee
81
82     def run_q43():
83         dates4, price4 = data.load_attach4()
84         dates, load, pv = data.load_attach2()
85         fc3 = data.load_attach3()
86         data.assert_aligned(dates4, dates, '附件4', '附件2')
87         rec = {}
88         E_start = C.E_INIT
89         for i in range(len(dates)):
90             pr = price4[i]
91             # 复用 main_q3 的 18.0 因果滚动模型（负荷预报化/实际SOC/去后见之明/光伏情景鲁棒）
92             res = _rolling_day(pr, load[i], pv[i], load[:i], pv[:i], fc3, dates[i], E_start)
93             res['E_start'] = E_start
94             rec[dates[i]] = res
95             E_start = res['E_end']
96         dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
97         P = {d: rec[d]['P'] for d in dates_out}
98         A = {d: rec[d]['A'] for d in dates_out}
99         Q = {d: rec[d]['Q'] for d in dates_out}
100        c = {d: rec[d]['c'] for d in dates_out}
101        f = {d: rec[d]['f'] for d in dates_out}
102        E0 = {d: rec[d]['E_start'] for d in dates_out}
103        Ee = {d: rec[d]['E_end'] for d in dates_out}
104        path = os.path.join(C.RES_DIR, 'result4-3.xlsx')
105        R.write_result3(path, dates_out, P, A, Q, c, f, E0, Ee, tpl_name='result4-3.xlsx')
106        tot_grid = sum(rec[d]['grid_fee'] for d in dates_out)
107        tot_emerg = sum(rec[d]['emerg_fee'] for d in dates_out)
108        print('已写出', path)
109        print(f'[Q4-3 波动电价+滚动MPC] 电网费={tot_grid:.2f} 紧急费={tot_emerg:.2f} 合计={tot_grid
        + tot_emerg:.2f} 元')

```

```

110 # 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月)
111 drow = [dict(date=str(d.date()), grid_fee=round(rec[d]['grid_fee'], 2),
112           emerg_fee=round(rec[d]['emerg_fee'], 2),
113           total=round(rec[d]['grid_fee'] + rec[d]['emerg_fee'], 2)) for d in dates]
114 pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q4_3_daily_cost.csv'),
115           index=False, encoding='utf-8-sig')
116 return dict(dates=dates, dates_out=dates_out, rec=rec,
117           tot_grid=tot_grid, tot_emerg=tot_emerg)
118
119
120 def plot_price_figure():
121     """图18: 附件4 波动电价(日均)曲线 vs 附件1 固定电价, 叠加季节分带与季节均值标注。
122     仅依赖原始数据(附件1/附件4), 不重新求解, 可独立调用。"""
123     dates4, price4 = data.load_attach4()
124     a1 = data.load_attach1()
125     mean_p = price4.mean(axis=1)
126     x = np.arange(len(mean_p))
127
128     def _seas(d):
129         m = d.month
130         return {12: '冬', 1: '冬', 2: '冬', 3: '春', 4: '春', 5: '春',
131                6: '夏', 7: '夏', 8: '夏', 9: '秋', 10: '秋', 11: '秋'}[m]
132     seas = np.array([_seas(d) for d in dates4])
133     SORDER = ['春', '夏', '秋', '冬']
134     SCOL = {'春': '#2ca02c', '夏': '#ff7f0e', '秋': '#9467bd', '冬': '#1f77b4'}
135     bounds = [i - 0.5 for i in range(1, len(seas)) if seas[i] != seas[i - 1]]
136
137     def _segments(sel):
138         """把选中日期的下标按连续性切成 (i0, i1) 段(冬季跨年需分两段)。"""
139         idx = np.where(sel)[0]
140         if len(idx) == 0:
141             return []
142         out, start = [], idx[0]
143         for a, b in zip(idx[:-1], idx[1:]):
144             if b != a + 1:
145                 out.append((start, a))
146                 start = b
147         out.append((start, idx[-1]))
148         return out
149
150     fig, ax = plt.subplots(figsize=(10.5, 4.9))
151     # 季节底色带(与表1"波动日电价"列对应; 冬季跨年分两段)
152     for s in SORDER:
153         for i0, i1 in _segments(seas == s):
154             ax.axvspan(i0 - 0.5, i1 + 0.5, color=SCOL[s], alpha=0.10)
155     ax.plot(x, mean_p, lw=1.3, color='k', label='附件4 波动电价(日均)')
156     p_fix = float(a1['price'].mean())

```

```

157 ax.axhline(p_fix, color='r', ls='--', lw=1.3,
158           label=f'附件1 固定电价（日均 {p_fix:.4f} 元/kWh）')
159 # 季节均值虚线 + 数值标注（= 表1 波动日电价）；冬季跨年分两段绘制
160 for s in SORDER:
161     mv = mean_p[seas == s].mean()
162     for i0, i1 in _segments(seas == s):
163         ax.hlines(mv, i0 - 0.5, i1 + 0.5, color=SCOL[s], ls=':', lw=1.8)
164         ax.text((i0 + i1) / 2, mv + 0.012, f'{s}季 {mv:.3f}', ha='center',
165               fontsize=11, color=SCOL[s],
166               bbox=dict(fc='white', ec='none', alpha=0.75, pad=0.2))
167 for b in bounds:
168     ax.axvline(b, color='gray', ls='--', lw=0.8, alpha=0.6)
169 ax.set_xlabel('日期（2025年1月1日起，按季分带）', fontsize=13)
170 ax.set_ylabel('平均电价(元/kWh)', fontsize=13)
171 ax.set_title('问题4：附件4波动电价（日均）与附件1固定电价对比', fontsize=14)
172 ax.legend(fontsize=11, loc='upper right', ncol=2); ax.grid(alpha=0.3)
173 ax.tick_params(labelsize=11)
174 fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q4_price.png'), dpi=150);
175     plt.close(fig)
176 print('图已存', os.path.join(C.FIG_DIR, 'q4_price.png'))
177
178 def shell():
179     q42 = run_q42()
180     q43 = run_q43()
181     plot_price_figure()
182
183
184 if __name__ == '__main__':
185     shell()

```